

BSc (Hons) in Computing (Information Systems)

ICT1303 – Database Systems II

1- Physical Database Tuning



ACKNOWLEDGEMENT

Presented by:

Ms. Hansika Samanthi

Contributors:

Ms. Hansika Samanthi

Mr. Mewan Jayathilake

Ms. Oshani Kaushalya

Reviewed by:

Mr. Janith Perera

RESOURCES

- Course page of the course
 - <https://lms.lnbt.lk/course/view.php?id=1536>
- References available for the course.
 - Hoffer, J. A., Venkataraman, R., & Topi, H. (2015). Modern Database Management, Global Edition (12th ed.). London, England: Pearson Education.
 - Elmasri, R., & Navathe, S. B. (2010). Fundamentals of database systems (6th ed.). Upper Saddle River, NJ: Pearson
- Online resources for the section
 - PostgreSQL Documentation: <https://www.postgresql.org/docs/current/sql-createprocedure.html>

AGENDA

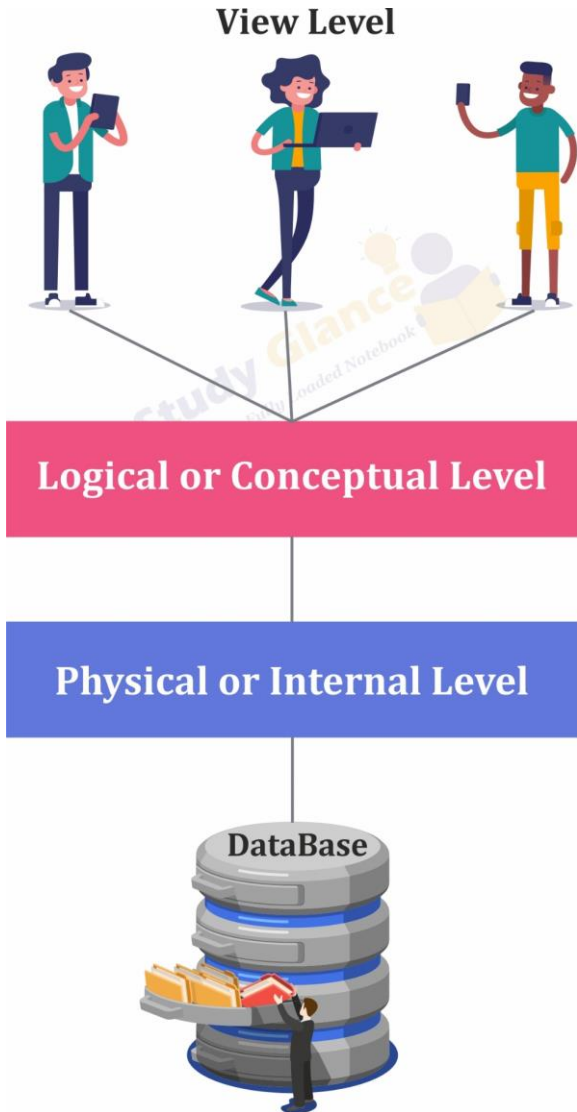
- 1.Database File
- 2.Row Size
- 3.Understanding Discrepancies in Row Size Calculation in PostgreSQL
- 4.Query Planner
- 5.Index Only Scan
- 6.Physical Database
- 7.File Structure
- 8.Physical Database Tuning
- 9.Indexing
 - 9.1. Primary Index
 - 9.2. Cluster Index
 - 9.3. Secondary Index
 - 9.4. Multi-level Indexing
 - 9.5. Mult-column Indexing
 - 9.6. Unused Indexes

INTENDED LEARNING OUTCOMES (ILO)

- By the end of this section, students should be able to:
- **ILO1:** Identify different types of indexes in PostgreSQL, including primary, cluster, and secondary indexes.
- **ILO2:** Explain the physical file structure of a database and how it impacts row size calculations.
- **ILO3:** Analyze the role of the query planner in optimizing database performance and the use of index-only scans.
- **ILO4:** Demonstrate the process of physical database tuning through efficient indexing strategies, including multi-level and multi-column indexing.
- **ILO5:** Evaluate the implications of unused indexes on database performance and storage efficiency.

1. DBMS Abstraction Levels

LEVELS OF ABSTRACTION



- Database systems include complex data structures.
- Regarding data retrieval, reducing complexity in terms of users' usability and making the system efficient, developers use levels of abstraction that hide irrelevant details from the users.
- Mainly there are three levels of abstraction for DBMS:
 1. Internal/Physical (Lowest Level)
 2. Conceptual/Logical
 3. External/User (Highest Level)

LEVELS OF ABSTRACTION

1. Internal/Physical

- Describes how the data is physically stored in the system.
- It includes details like file structures, storage formats, indexing methods, and memory allocation (binary, hexadecimal, etc.)



2. Conceptual/Logical –

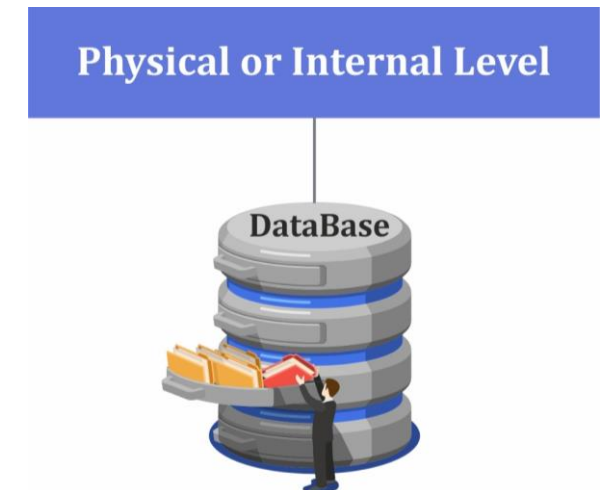
- Defines what data is stored and the relationships between different data entities.
- It represents the database structure (e.g., tables, constraints, and schemas) without worrying about physical implementation.

Logical or Conceptual Level

LEVELS OF ABSTRACTION

3. External/User

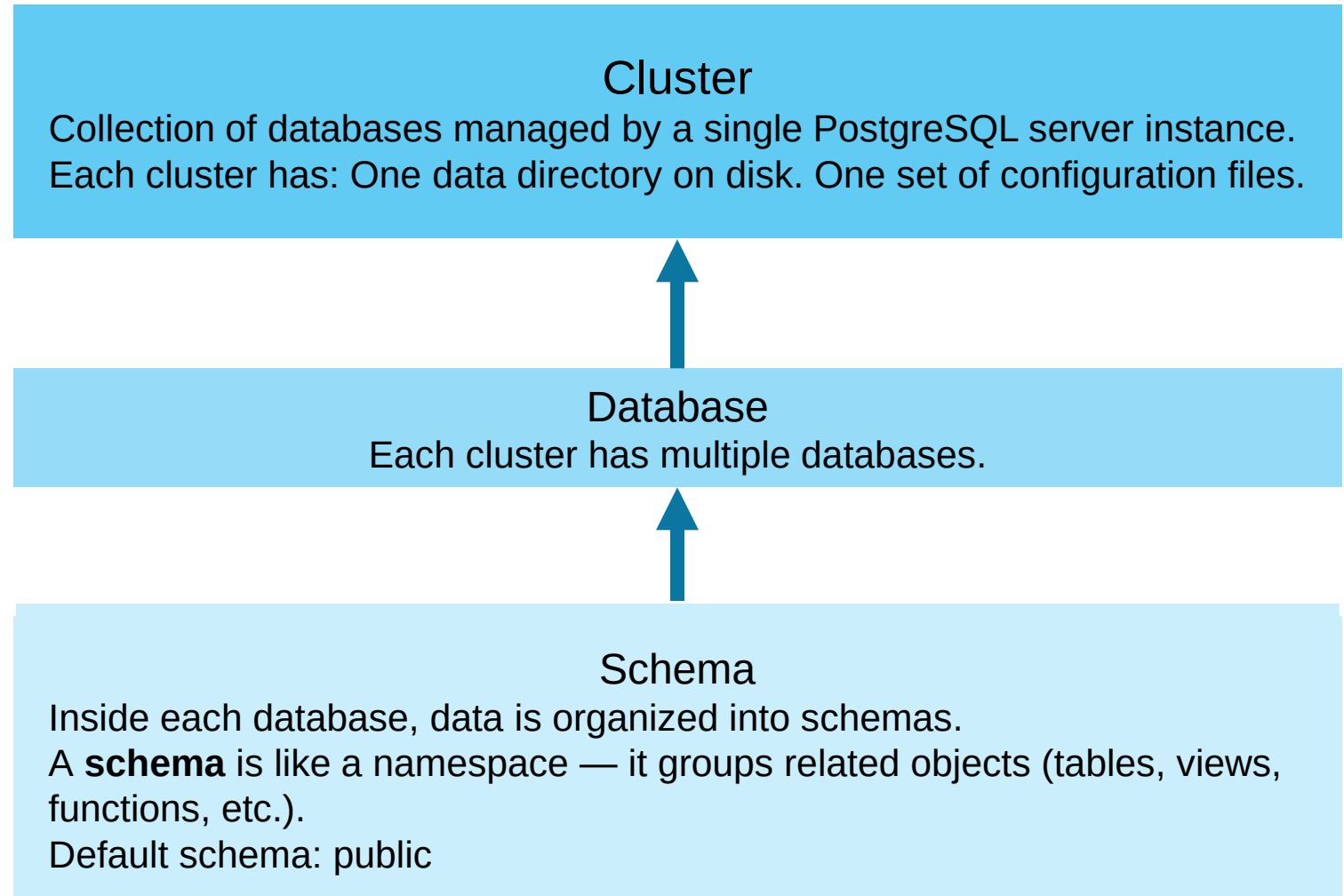
- This is the highest level of abstraction, representing how users or applications interact with the data.
- It includes customized views, query results, and interfaces tailored for different users, hiding complex underlying structures.



2. Data Storage Hierarchy in PostgreSQL

PostgreSQL - Data Storage Hierarchy

- PostgreSQL data storage hierarchy is slightly different (and more structured) than in some other systems.
- Here's how PostgreSQL organizes its data — from top to bottom:



PostgreSQL - LOCATE THE DATA DIRECTORY

- PostgreSQL stores its database files in a structured directory hierarchy within the data directory. (Usually found at `C:\Program Files\PostgreSQL<version>\data\`)
- Also, the location of this directory also can be found by executing the following SQL command in your **PostgreSQL client**.

SQL Command



```
postgres=# show data_directory;
data_directory
-----
C:/Program Files/PostgreSQL/16/data
(1 row)
```

PostgreSQL - LOCATE THE DATA DIRECTORY

C:/Program Files/PostgreSQL/17/data

This PC > Local Disk (C:) > Program Files > PostgreSQL > 16 > data			
Name	Date modified	Type	Size
base	10/15/2024 8:54 AM	File folder	
global	10/15/2024 9:06 AM	File folder	
log	10/14/2024 8:34 AM	File folder	
pg_commit_ts	9/25/2024 11:35 PM	File folder	
pg_dynshmem	9/25/2024 11:35 PM	File folder	
pg_logical	10/15/2024 9:59 AM	File folder	
pg_multixact	9/25/2024 11:35 PM	File folder	
pg_notify	9/25/2024 11:35 PM	File folder	
pg_replslot	9/25/2024 11:35 PM	File folder	
pg_serial	9/25/2024 11:35 PM	File folder	
pg_snapshots	9/25/2024 11:35 PM	File folder	
pg_stat	9/25/2024 11:35 PM	File folder	
pg_stat_tmp	9/25/2024 11:35 PM	File folder	
pg_subtrans	9/25/2024 11:35 PM	File folder	
pg_tblspc	9/25/2024 11:35 PM	File folder	
pg_twophase	9/25/2024 11:35 PM	File folder	
pg_wal	10/14/2024 5:01 PM	File folder	
pg_xact	9/25/2024 11:35 PM	File folder	
current_logfiles	10/15/2024 11:11 AM	File	1 KB
pg_hba.conf	9/25/2024 11:35 PM	CONF File	6 KB
pg_ident.conf	9/25/2024 11:35 PM	CONF File	3 KB
1 item selected			

ACTIVITY 1

- Create a database named 'companyDb' using psql.

```
CREATE DATABASE companyDb;
```

- Connect to the new database

```
\c companyDb;
```

- Create the Department table as follows:

```
CREATE TABLE department (  
    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    name VARCHAR(100) NOT NULL  
);
```

- Insert data to the Department table.

```
INSERT INTO department (name) VALUES ('Academic'), ('Administration');
```

ACTIVITY 1

- Create the Designation table.

```
CREATE TABLE designation (  
    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    department_id INT NOT NULL,  
    FOREIGN KEY (department_id) REFERENCES department(id)  
);
```

- Insert data to the Designation table.

```
INSERT INTO designation (name, department_id) VALUES ('Lecturer', 1),  
  
('Instructor', 1), ('Registrar', 2);
```

ACTIVITY 1

- Insert more data to the Designation table using a for loop.

```
DO $$  
DECLARE  
    i INT;  
BEGIN  
    FOR i IN 1..5 LOOP  
        INSERT INTO designation (name, department_id)  
        SELECT name, department_id FROM designation;  
    END LOOP;  
END $$;
```


ACTIVITY 1

- Create employee table.

```
CREATE TABLE Employee (  
    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    nic VARCHAR(20) NOT NULL UNIQUE,  
    designation_id INT NOT NULL,  
    FOREIGN KEY (designation_id) REFERENCES Designation(id)  
);
```

- Insert data to the employee table.

```
INSERT INTO Employee (name, nic, designation_id) VALUES  
( 'Sarah Johnson', '871234567V', 1),  
( 'David Wilson', '901234567V', 3);
```

ACTIVITY 1

- Insert more data to the employee table.

```
CREATE SEQUENCE IF NOT EXISTS nic_seq START WITH 1000000000;  
  
DO $$  
DECLARE  
    i INT;  
BEGIN  
    FOR i IN 1..10 LOOP  
        INSERT INTO employee (name, nic, designation_id)  
        SELECT name, nextval('nic_seq') || 'V', designation_id FROM  
employee;  
    END LOOP;  
END $$;
```

PostgreSQL – VIEW DATABASE

- To show the databases in PostgreSQL, the following several methods can be used depending on the environment.
 1. **SQL Query:** Use `SELECT datname FROM pg_database;`
 2. **SQL Command:** Use `\l`
 3. **pgAdmin:** Navigate through the graphical interface.

1. SQL QUERY

```
postgres=# SELECT datname from pg_database;
```

```
datname
```

```
-----
```

```
postgres  
template1  
template0  
abx  
abcsupermarket  
bankdb  
companydatabase  
company_db  
company  
BakeShop  
(10 rows)
```

- pg_database is a system catalog that lives inside every PostgreSQL cluster.
- It contains one row per database in the cluster.
- Each row describes metadata about a single database — like its name, owner, encoding, tablespace, and more.

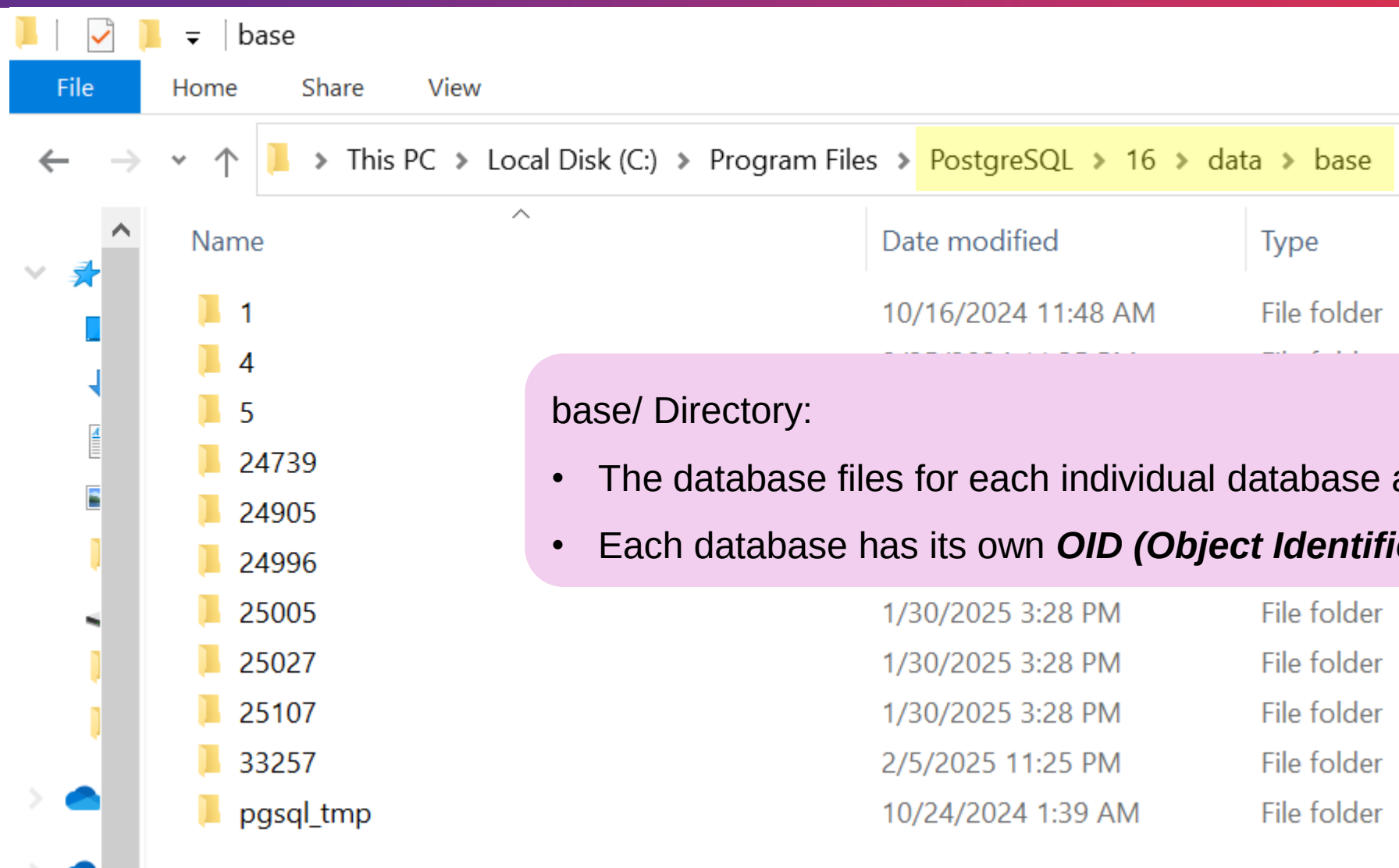
2. SQL COMMAND

```
postgres=# \l
```

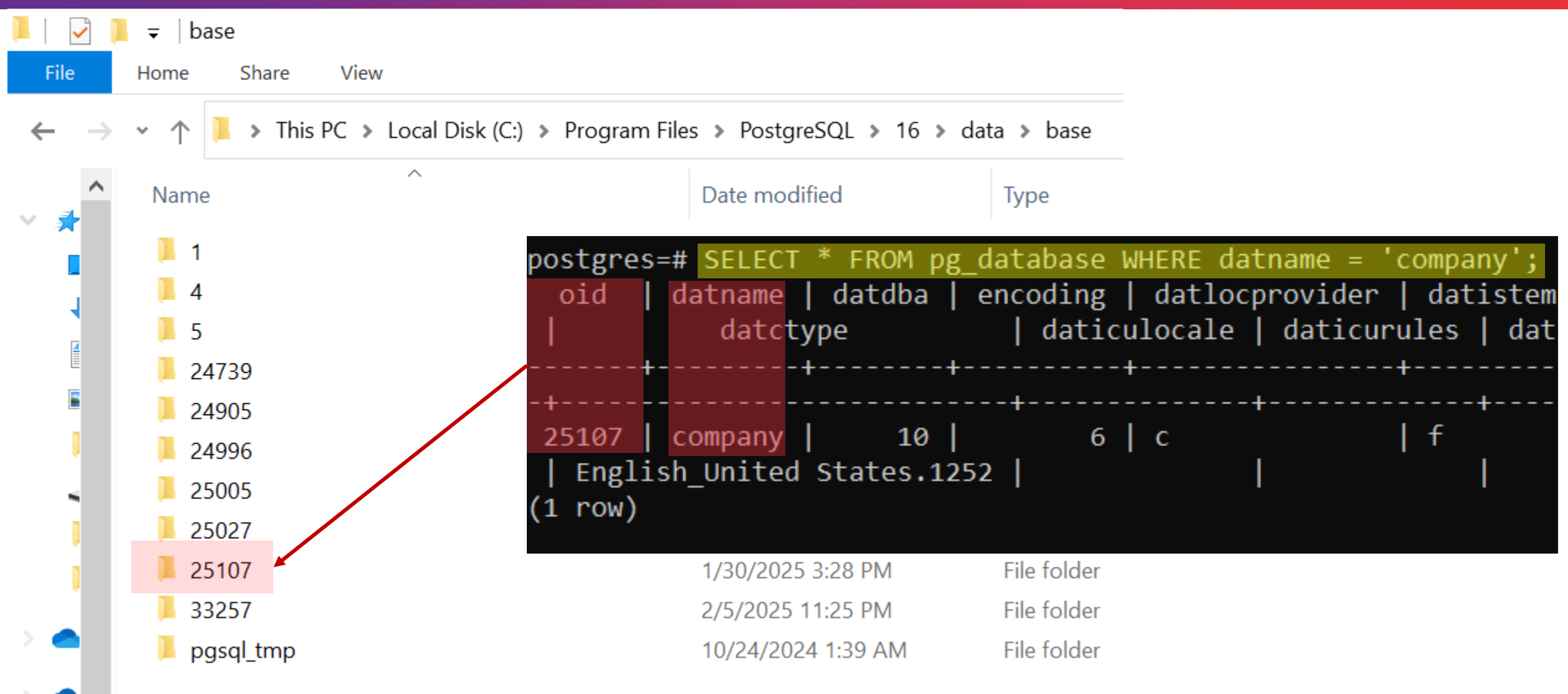
List of databases										
Name	Owner	Encoding	Locale Provider	Collate		Ctype		ICU Locale	ICU Rules	Access privileges
BakeShop	postgres	UTF8	libc	English_United	States.1252	English_United	States.1252			
abcsupermarket	postgres	UTF8	libc	English_United	States.1252	English_United	States.1252			
abx	postgres	UTF8	libc	English_United	States.1252	English_United	States.1252			
bankdb	postgres	UTF8	libc	English_United	States.1252	English_United	States.1252			
company	postgres	UTF8	libc	English_United	States.1252	English_United	States.1252			=Tc/postgres + postgres=CTc/postgres+ admin=C/postgres
company_db	postgres	UTF8	libc	English_United	States.1252	English_United	States.1252			
companydatabase	postgres	UTF8	libc	English_United	States.1252	English_United	States.1252			
postgres	postgres	UTF8	libc	English_United	States.1252	English_United	States.1252			
template0	postgres	UTF8	libc	English_United	States.1252	English_United	States.1252			=c/postgres + postgres=CTc/postgres

```
-- More --
```

FINDING DATABASE FILES INSIDE THE DATA DIRECTORY



FINDING DATABASE FILES INSIDE THE DATA DIRECTORY



File Explorer window showing the directory path: This PC > Local Disk (C:) > Program Files > PostgreSQL > 16 > data > base.

The file list on the left shows folders named 1, 4, 5, 24739, 24905, 24996, 25005, 25027, 25107, 33257, and pgsql_tmp. The folder 25107 is highlighted in pink, and a red arrow points from it to a terminal window.

The terminal window shows the command: `postgres=# SELECT * FROM pg_database WHERE datname = 'company';`

The output of the command is a table with columns: `oid`, `datname`, `datdba`, `encoding`, `datlocprovider`, `datistemp`, `datctype`, `daticulocale`, and `daticurules`. The output shows one row for the 'company' database with `oid` 25107.

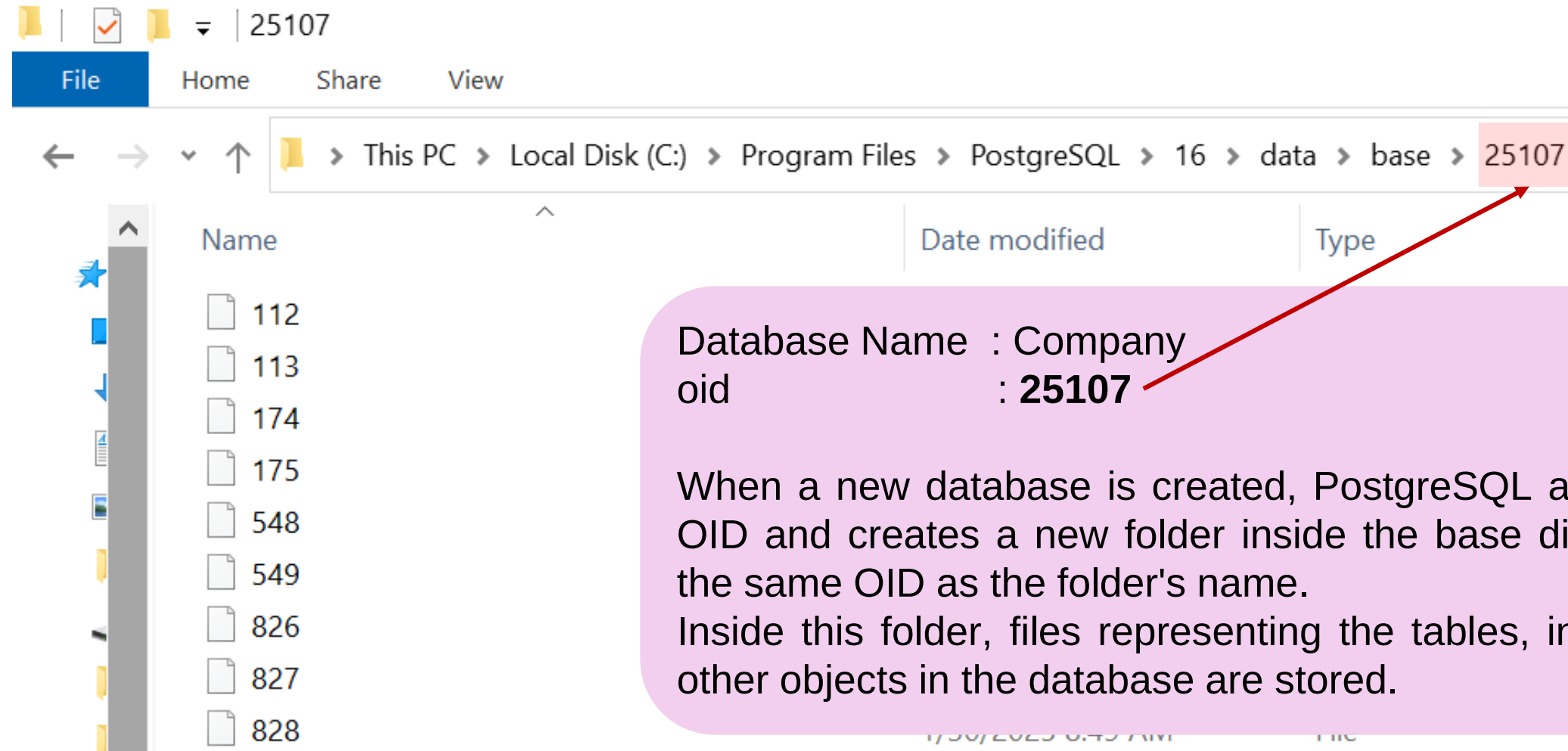
oid	datname	datdba	encoding	datlocprovider	datistemp	datctype	daticulocale	daticurules
25107	company	10	6	c	f	English_United States.1252		

The table output is shown in a terminal window with the following columns: `oid`, `datname`, `datdba`, `encoding`, `datlocprovider`, `datistemp`, `datctype`, `daticulocale`, and `daticurules`. The output shows one row for the 'company' database with `oid` 25107.

The file list on the right shows the following folders and their dates modified and types:

Name	Date modified	Type
1	1/30/2025 3:28 PM	File folder
4	2/5/2025 11:25 PM	File folder
5	10/24/2024 1:39 AM	File folder
24739		
24905		
24996		
25005		
25027		
25107		
33257		
pgsql_tmp		

INTERNAL VIEW OF DATABASE FILE



3. Table Statistics

TABLE STATISTICS

- The following query is used to retrieve statistical information about user tables in PostgreSQL.

```
postgres=# SELECT * FROM pg_catalog.pg_stat_user_tables;
```

- **pg_catalog** is a special schema in PostgreSQL that stores information about the database itself.
- **pg_stat_user_tables** is a view in PostgreSQL that provides important statistics about the tables created by users (as opposed to the system tables).

NOTE:

- *Schema in PostgreSQL is like a folder that organizes and contains database objects (such as tables, views, and functions).*
- *pg_catalog is the default schema where PostgreSQL keeps its system-related information.*

TABLE STATISTICS

Key columns in `pg_stat_user_tables`

- `relname` : Table name.
- `seq_scan` : Number of sequential scans on the table.
- `seq_tup_read` : Number of tuples (rows) read by sequential scans.
- `idx_scan` : Number of index scans on the table.
- `n_tup_ins` : Number of rows inserted.
- `n_tup_upd` : Number of rows updated.
- `n_tup_del` : Number of rows deleted.
- `n_live_tup` : Number of live rows (currently in use).
- `n_dead_tup` : Number of dead rows (marked for deletion).

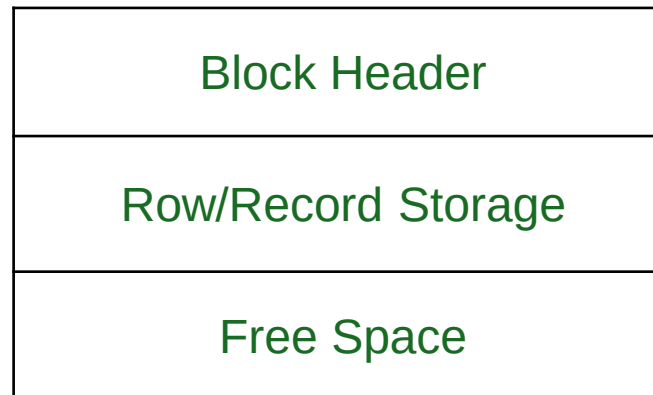
- **Performance Monitoring:** Helps in understanding how your tables are being accessed (e.g., through sequential or index scans).
- **Table Maintenance:** Provides insight into rows being inserted, updated, or deleted over time.
- **Vacuuming Decisions:** Use dead/live row stats to determine when to perform VACUUM operations to clean up dead tuples.

4. Table Block in PostgreSQL

WHAT IS BLOCK SIZE IN PostgreSQL?

- **Block Size** refers to the size of a single data block in PostgreSQL, which is the smallest unit of storage that the DBMS reads/ write from disk.
- The default block size is typically **8192 bytes** (8 KB), which can be configured at the time of database initialization.

PostgreSQL Block (8 KB)



Example

- Even if we add 2 more rows to the final block, the size of the table (Employee) will not differ (24 KB).
- But, as soon as the next row (10th row) is added the table size would change into 32 KB.
- The no of rows in a block can differ accordingly.

Employee Table

EmployeeId	Name	DepartmentId

PostgreSQL Block (8 KB)

Block Header
Row/Record Storage
Free Space

ACTIVITY 2

- Observe the block size of the database management system.

```
SHOW block_size;
```

```
block_size
-----
8192
```

- Find the average column sizes in the employee table:

```
SELECT
```

```
    AVG(pg_column_size(id)),
```

```
    AVG(pg_column_size(name)),
```

```
    AVG(pg_column_size(nic)),
```

```
    AVG(pg_column_size(designation_id))
```

```
FROM employee;
```

avg	avg	avg	avg
4.0000000000000000	13.5000000000000000	11.0000000000000000	4.0000000000000000

- Calculate the row size .

$$\text{Row size} = 4 + 13.5 + 11 + 4 = 32.5 \text{ B}$$

ACTIVITY 2

- Find the blocking factor using the following statement.

```
SELECT
    trim(both '(' from split_part(ctid::text, ',', 1))::int AS block_number,
    COUNT(*) AS row_count
FROM Employee
GROUP BY block_number
ORDER BY block_number;
```

- Calculate the blocking factor.

$$\text{Blocking Factor (employee)} = \left\lfloor \frac{\text{Block Size}}{\text{Row Size}} \right\rfloor = \left\lfloor \frac{8192 \text{ B}}{32.5 \text{ B}} \right\rfloor = 252$$

Note: This is different from the actual value 127 as row size should include hidden rows.

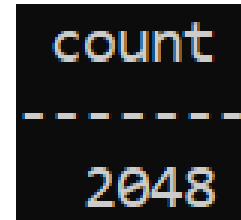
block_number	row_count
0	127
1	127
2	127
3	127
4	127
5	127
6	127
7	127
8	127
9	127
10	127
11	127
12	127
13	127
14	127
15	127
16	16

(17 rows)

ACTIVITY 2

- Find the number of rows in employee table.

```
select count(*) from employee;
```



```
count
-----
2048
```

- Find the number of blocks.

$$\# \text{ blocks (employee)} = \left\lceil \frac{\# \text{ rows}}{\text{Blocking Factor}} \right\rceil = \left\lceil \frac{2048}{252} \right\rceil = 9$$

Note: This is different from the actual value 17 as row size does not include hidden rows.

- View the indexes in the employee table.

```
\d employee;
```

- Drop the primary index from the table.

```
DROP INDEX employee_pkey;
```

- Retrieve the cost for selecting a particular employee.

```
EXPLAIN SELECT * FROM employee WHERE id = 25;
```

ACTIVITY 2

- Calculate the I/O cost for the query.
 - This employee table is ordered by primary key (id). And it has no duplicate values. Therefore, binary search can be performed.

$$Cost = \lfloor \log_2(\# \text{ blocks}) \rfloor = \lfloor \log_2(9) \rfloor = \lfloor 3. \rfloor = 4 \text{ IO}$$

ACTIVITY 2

employee			
id (4 B)	name (13.5 B)	Nic (11 B)	designation_id (4 B)
101	Lecturer		
102	Clerk		
103	Manager		
107	Demonstrator		
109	Researcher		
251	Registrar		
259	Assistant Registrar		
261	Security Officer		

Block 1 (employee) – 252 rows

Block 9 (employee)

ACTIVITY 2

- Let us introduce a primary index to this table.

Index Table	
id (4 B)	Pointer (4 B)

- Calculate the row size .

$$\text{Row size} = 4 + 4 = 8 \text{ B}$$

- Calculate the blocking factor.

$$\text{Blocking Factor (PI of employee)} = \left\lfloor \frac{\text{Block Size}}{\text{Row Size}} \right\rfloor = \left\lfloor \frac{8192 \text{ B}}{8 \text{ B}} \right\rfloor = 1024$$

- Find the number of blocks.

$$\# \text{ blocks (PI of employee)} = \left\lceil \frac{\# \text{ rows}}{\text{Blocking Factor}} \right\rceil = \left\lceil \frac{\# \text{ blocks in employee table}}{\text{blocking factor}} \right\rceil = \left\lceil \frac{9}{1024} \right\rceil = \lceil 0.008 \rceil = 1$$

ACTIVITY 2

Index Table	
id (4 B)	Pointer (4 B)
101	
110	
Block 1 – 1024 rows	

employee			
id (4 B)	name (13.5 B)	Nic (11 B)	designation_id (4 B)
101	Lecturer		
102	Clerk		
103	Manager		
107	Demonstrator		
109	Researcher		
110			
251	Registrar		
259	Assistant Registrar		
261	Security Officer		

ACTIVITY 2

Let us introduce a primary index to this table.

- Designate id as the primary key for the employee table. This will automatically create a primary index for the employee table.

```
ALTER TABLE employee ADD PRIMARY KEY (id);
```

- Retrieve the cost for selecting a particular employee.

```
EXPLAIN SELECT * FROM employee WHERE id = 25;
```

- Observe that the second query uses the primary index resulting a low cost.
- Calculate the I/O cost for the query.
 - This PI of employee table is ordered. And it has no duplicate values. Therefore, binary search can be performed.

$$\text{Cost} = \text{cost on index} + \text{cost on base table} = \lfloor \log_2(\# \text{ blocks}) \rfloor + 1 = \lfloor \log_2(1) \rfloor + 1 = \lfloor 0 \rfloor + 1 = 1 \text{ IO}$$

Block Factor – PostgreSQL

- Block factor in a table is number of rows that can be accommodated into a block.
- In example, the block factor of the Employee table is 3.

Employee Table

EmployeeId	Name	DepartmentId

Block 1
(3 Rows)

Block 2
(3 Rows)

Block 3
(1 Row)

$$\text{Block Factor (BF)} = \frac{\text{Block Size (BS)}}{\text{Row Size}}$$

FIND BLOCK SIZE IN PostgreSQL

To find the block size of your PostgreSQL database, you can execute the following SQL command,

```
show block_size;
```

This will return the block size in bytes as below.

```
postgres=# show block_size;
 block_size
-----
    8192
(1 row)
```


FIND TABLE SIZE - PostgreSQL

The following query retrieves the size of the **employee** table.

```
SELECT
table_name,
pg_size_pretty(pg_relation_size('employee'))
AS total_size
FROM information_schema.tables
WHERE table_name = 'employee'
AND table_schema = 'public';
```

pg_size_pretty();

- Converts the result of pg_total_relation_size() (in bytes) into a more readable format (e.g., KB, MB).

pg_total_relation_size('public.employee');

- Returns the total size of the table employee, including its indexes and any other related overhead.

pg_relation_size('public.employee');

- Determine the disk space used by 'employee' table.

FIND TABLE SIZE - PostgreSQL

```
company=# select table_name,  
company-# pg_size_pretty(pg_relation_size('employee'))  
company-# AS pretty_size  
company-# FROM information_schema.tables  
company-# WHERE table_name = 'employee'  
company-# AND table_schema = 'public';  
table_name | pretty_size  
-----+-----  
employee   | 8192 bytes  
(1 row)
```

```
company=# select table_name,  
company-# pg_total_relation_size('employee')  
company-# AS total_relation_size  
company-# FROM information_schema.tables  
company-# WHERE table_name = 'employee'  
company-# AND table_schema = 'public';  
table_name | total_relation_size  
-----+-----  
employee   | 24576  
(1 row)
```

```
company=# select table_name,  
company-# pg_relation_size('employee')  
company-# AS relation_size  
company-# FROM information_schema.tables  
company-# WHERE table_name = 'employee'  
company-# AND table_schema = 'public';  
table_name | relation_size  
-----+-----  
employee   | 8192  
(1 row)
```

CALCULATE NUMBER OF BLOCKS - Manual

To calculate the number of blocks given:

- Table size = 80 KB
- Block size = 8 KB

Use the following formula:

$$\text{Number of blocks} = \frac{\text{Table Size}}{\text{Block Size}}$$

Substituting the given values:

$$\text{Number of blocks} = \left\lceil \frac{80 \text{ KB}}{8 \text{ KB}} \right\rceil = 10$$

So, the employee table would occupy 10 blocks.

5. Row (Record) Size

AVERAGE ROW SIZE OF A TABLE

In PostgreSQL, the average row size in a table depends on several factors, including:

- Column data types (fixed vs. variable length)
- NULL values (stored as a bitmap, not taking actual space)
- TOAST (The Oversized-Attribute Storage Technique) for large values
- Alignment padding (PostgreSQL aligns data types to word boundaries)

FIND AVERAGE ROW SIZE OF A TABLE - PostgreSQL

Method 1: Average Size of All Rows

```
SELECT AVG (pg_column_size(employee.*)) AS average_row_size FROM employee;
```

Method 2: Using block size and average number of records per block.

$$\frac{\text{Block Size}}{\text{Average (Number of records per block)}}$$

Method 3: Total of average column sizes in the table

```
-- Column size of id in employee table  
SELECT AVG(pg_column_size(id)) FROM employee ;
```

$$\sum \text{AVG}(\text{COLUMN SIZE 1} + \text{COLUMN SIZE 2} + \text{COLUMN SIZE 3})$$

METHOD 1: AVERAGE SIZE OF ALL ROWS

```
company=# SELECT AVG(pg_column_size(employee.*)) AS average_row_size FROM employee;  
average_row_size  
-----  
60.4882352941176471  
(1 row)
```

FIND BLOCK FACTOR - PostgreSQL

- SQL query designed to retrieve information about the number of rows in the Employee table, grouped by the block number. Below is a breakdown of the query.

```
SELECT
    trim(both '(' from split_part(ctid::text, ',', 1))::int AS block_number,
    COUNT(*) AS row_count
FROM Employee
GROUP BY block_number
ORDER BY block_number;
```

block_number	row_count
0	119
1	51

(2 rows)

NOTE

- ctid Column:** Stores the physical location of a row in the format (block_number, tuple_index).

Extracting Block Number:

- split_part(ctid::text, ',', 1)** : Gets the first part (block number).
- trim(both '(' ...)::int** : Removes (and converts to an integer.

WHAT IS ctid?

Meaning of ctid

- **Physical Location:** ctid indicates the physical location of a row within its table in terms of the block and offset within that block. Each row in a PostgreSQL table is stored on a specific page (block) of the database.
- **Structure:** The ctid is represented as a combination of two values:
 - **Block Number:** The number of the block (page) in which the row is stored.
 - **Tuple Index:** The index of the row within that block.

For example, a ctid value might look like (0, 123), where 0 is the block number and 123 is the index of the row in that block.



WHAT IS ctid ...

Use Cases for ctid

- **Debugging and Troubleshooting:** ctid can help identify the physical location of a row when debugging issues related to data corruption or performance.
- **Row Identification:** It can be used in situations where you need to reference a specific row when no unique identifier is available.
- **Performance Optimization:** Since ctid indicates the physical location, it can sometimes be used in optimization strategies for certain types of queries.



FIND Ctid - PostgreSQL

```
company=# SELECT *,CTID FROM employee;
```

employee_id	name	gender_id	designation_id	salary	ctid
1	Alice Fernando	1	101	75000.00	(0,1)
2	Bob Silva	1	102	92000.50	(0,2)
3	Carol Perera	2	103	68000.00	(0,3)
4	David Kumar	1	101	77000.00	(0,4)
5	Eva Jayasinghe	2	104	60000.00	(0,5)
6	Frank Wijesinghe	1	105	71000.25	(0,6)
7	Grace De Silva	2	102	95000.00	(0,7)
8	Henry Gunasekara	1	106	66000.75	(0,8)
9	Isabella Fernando	2	103	69000.00	(0,9)
10	John Ratnayake	1	107	105000.00	(0,10)
11	Kavindu Perera	1	101	72000.00	(0,11)
12	Nadeesha Silva	2	104	63000.00	(0,12)
13	Chathura Fernando	1	107	110000.00	(0,13)
14	Sajani Wijeratne	2	103	67000.00	(0,14)
15	Ravindu Gunawardena	1	106	68000.00	(0,15)
16	Thilini Dissanayake	2	105	70000.00	(0,16)
17	Kasun Jayasuriya	1	102	93000.00	(0,17)
18	Dilani Senanayake	2	103	69500.00	(0,18)
19	Suresh Samarasinghe	1	101	76000.00	(0,19)
20	Anushka Madushani	2	104	62000.00	(0,20)
21	Alexandra Jones	2	207	106921.94	(0,21)
22	Joshua Chavez	1	201	105492.56	(0,22)
23	Jennifer Morgan	1	202	62100.23	(0,23)
24	Joshua Good	1	201	97456.74	(0,24)
25	Holly Simmons	2	201	95066.24	(0,25)
26	Kevin Navarro	1	204	92768.09	(0,26)
27	Mathew Clarke	2	204	95617.88	(0,27)

-- More --

CALCULATE THE RECORD SIZE

To calculate the record size, given:

- Block size = 8KB = 8192 B
- Number of records per block = 119

You can use the following formula:

$$\text{Record size} = \frac{\text{Block Size}}{\text{Number of records per block}}$$

Substituting the given values:

$$\text{Record size} = \frac{8192 \text{ B}}{119} = 68.840 \text{ B}$$

So, each record is approximately 68.84 B

FIND SIZE OF A ROW USING ctid

Example: Block 0, row 1 (ctid (0, 1))

```
company=# SELECT pg_column_size(t) AS row_size FROM employee t WHERE ctid = '(0,1)';
 row_size
-----
      59
(1 row)
```

employee_id	name	gender_id	designation_id	salary	ctid
1	Alice Fernando	1	101	75000.00	(0,1)
2	Bob Silva	1	102	92000.50	(0,2)
3	Carol Perera	2	103	68000.00	(0,3)
4	David Kumar	1	101	77000.00	(0,4)
5	Eva Jayasinghe	2	104	60000.00	(0,5)
6	Frank Wijesinghe	1	105	71000.25	(0,6)
7	Grace De Silva	2	102	95000.00	(0,7)
8	Henry Gunasekara	1	106	66000.75	(0,8)
9	Isabella Fernando	2	103	69000.00	(0,9)
10	John Ratnayake	1	107	105000.00	(0,10)
11	Kavindu Perera	1	101	72000.00	(0,11)
12	Nadeesha Silva	2	104	63000.00	(0,12)
13	Chathura Fernando	1	107	110000.00	(0,13)
14	Sajani Wijeratne	2	103	67000.00	(0,14)
15	Ravindu Gunawardena	1	106	68000.00	(0,15)
16	Thilini Dissanayake	2	105	70000.00	(0,16)
17	Kasun Jayasuriya	1	102	93000.00	(0,17)

METHOD 3: TOTAL OF AVERAGE COLUMN SIZES IN THE TABLE

- To calculate the average row size for the Employee table, we individually compute the size of each column using the **pg_column_size()** function.
- **pg_column_size(column_name)**: Returns the size (in bytes) of the specified column for each row.

```
SELECT AVG(pg_column_size(id)) FROM employee;  
SELECT AVG(pg_column_size(name)) FROM employee;  
SELECT AVG(pg_column_size(nic)) FROM employee;  
SELECT AVG(pg_column_size(designation_id)) FROM employee;
```

CALCULATING THE TOTAL AVERAGE ROW SIZE

Query Results

employee_id: 4 B

name: 14.311 B

gender_id: 4 B

designation_id: 4 B

salary: 4 B

To calculate the total average row size, we sum the sizes of all columns:

$$\sum AVG(COLUMN\ SIZE\ 1 + COLUMN\ SIZE\ 2 + COLUMN\ SIZE\ 3 + COLUMN\ SIZE\ 4 + COLUMN\ SIZE\ 5)$$

$$\text{Total Row Size} = 4 + 14.311 + 4 + 4 + 4 = 30.311\text{ B}$$

- The average row size for the Employee table is **30.31** bytes.
- This calculation is useful for understanding storage requirements and optimizing database performance.

ROW SIZE IN PostgreSQL

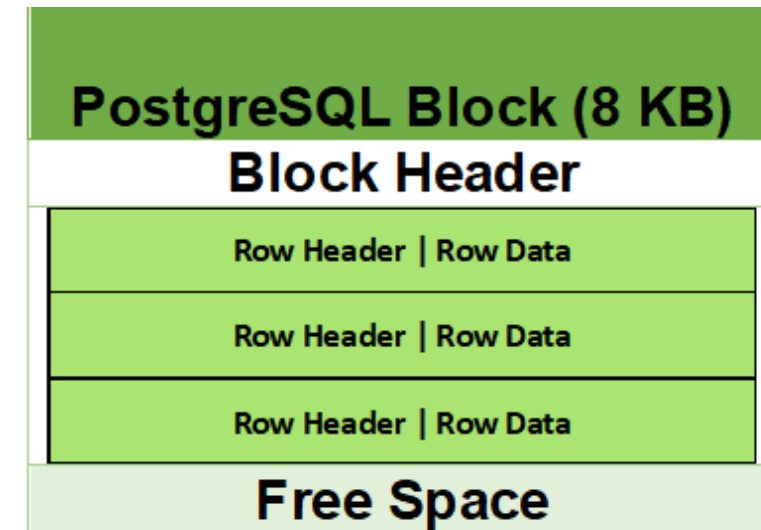
- The size of a row (also known as a tuple) in PostgreSQL can vary depending on:
 - The data types used in the table
 - The specific values stored

Row Size Limit

- Theoretically, a single row can be up to 1.6 TB. In practice, the row size is limited by:
 - The block size (typically 8 KB)
 - Storage overhead
- In an 8 KB block, the actual row size is restricted by the space left after:
 - The block header
 - Free space.

TOAST (The Oversized-Attribute Storage Technique)

- If a row exceeds the block size limit (for example, due to large text, bytea, or other large types):
 - PostgreSQL uses TOAST.
 - TOAST stores large values outside the block in separate storage.
 - The main row only keeps a reference to the large value



Understanding Discrepancies in Row Size Calculation in PostgreSQL

AVERAGE ROW SIZE – Using Query

```
SELECT AVG(pg_column_size(employee_id)) FROM employee;  
SELECT AVG(pg_column_size(name)) FROM employee;  
SELECT AVG(pg_column_size(gender_id)) FROM employee;  
SELECT AVG(pg_column_size(designation_id)) FROM employee;  
SELECT AVG(pg_column_size(salary)) FROM employee;
```

Total Calculated Average Row Size: **30.31 B**

Whole Row Calculation Using,

```
SELECT AVG(pg_column_size(Employee)) AS average_row_size FROM Employee;
```

Resulted Row Size: **60.48 B**

AVERAGE ROW SIZE – CALCULATED

- To calculate the record size given:
- Block Size = 8 KB = 8192 B
- Number of records per block = 119

$$\text{Record Size} = \frac{\text{Block Size}}{\text{Number of records per block}}$$

- Substituting the given values:

$$\text{Record Size} = \frac{8192}{119} = 68.84 \text{ B}$$

WHY THE DISCREPANCY

Overhead Costs

- Whole Row Calculation (``Employee.``) includes additional storage overhead:
- Tuple Header: Typically, around 24 bytes, which includes metadata like transaction information, tuple identifiers, etc.
- Alignment Padding: PostgreSQL aligns data for performance reasons, adding padding bytes between columns.
- These overheads are not included in the column-by-column calculations, which only account for raw column data sizes.

BREAKDOWN OF OVERHEAD COSTS

Tuple Header: ~24 B

Alignment Padding: Can vary depending on data types and their alignment requirements.

Example:

Calculated Row Size (Columns) = 30.11 B

Whole Row Size (including overhead) = 60.48 B

Overhead = $60.48 - 30.11 \text{ B} = 30.17 \text{ B}$

This value aligns with the typical overhead for tuple headers and padding.

NOTE:

Column-by-Column Calculation: Only sums the raw data sizes.

Whole Row Calculation (Employee.*) :

- Includes additional overhead such as tuple headers and padding.
- Understanding this difference is important when estimating storage needs and optimizing database design.

INTRODUCTION TO EFFICIENT DATA RETRIEVAL

How do databases retrieve data?

Databases use search queries to fetch specific records.

```
SELECT FirstName FROM employee WHERE EmpID = 3;
```

Challenge

What happens when the database has millions of rows?

UNDERSTANDING TABLE STRUCTURE

Typical Table Layout:

Example: Employee Table

Ctid	EmpID	FirstName	LastName	DepartmentID
(01)	1	A	Perera	1
(02)	2	B	Silva	2
(03)	3	C	Fernando	1

Activity



How does the database find the record you want in a large table?

1

Query Planner

QUERY PLANNER AKA QUERY OPTIMIZER

- A query planner (or query optimizer) is a component of a database management system (DBMS) responsible for determining the most efficient way to execute a given SQL query.
- The planner analyzes the query and decides on the best execution plan based on various factors, including available indexes, statistics about the data, and the query's structure.
- In PostgreSQL, you can view the query planner's execution plan using the **EXPLAIN** statement.

Ex:

```
EXPLAIN SELECT * FROM employee WHERE designation_id = 103;
```

- *This command will return the execution plan for the query.*
- *It will show the type of scan used, the estimated number of rows and the cost of the operation to retrieve rows from the employee table where designation_id is 3.*
- *It won't return the actual data from the query itself.*

EXMAPLE

```
company=# EXPLAIN SELECT * FROM employee WHERE designation_id = 103;  
QUERY PLAN
```

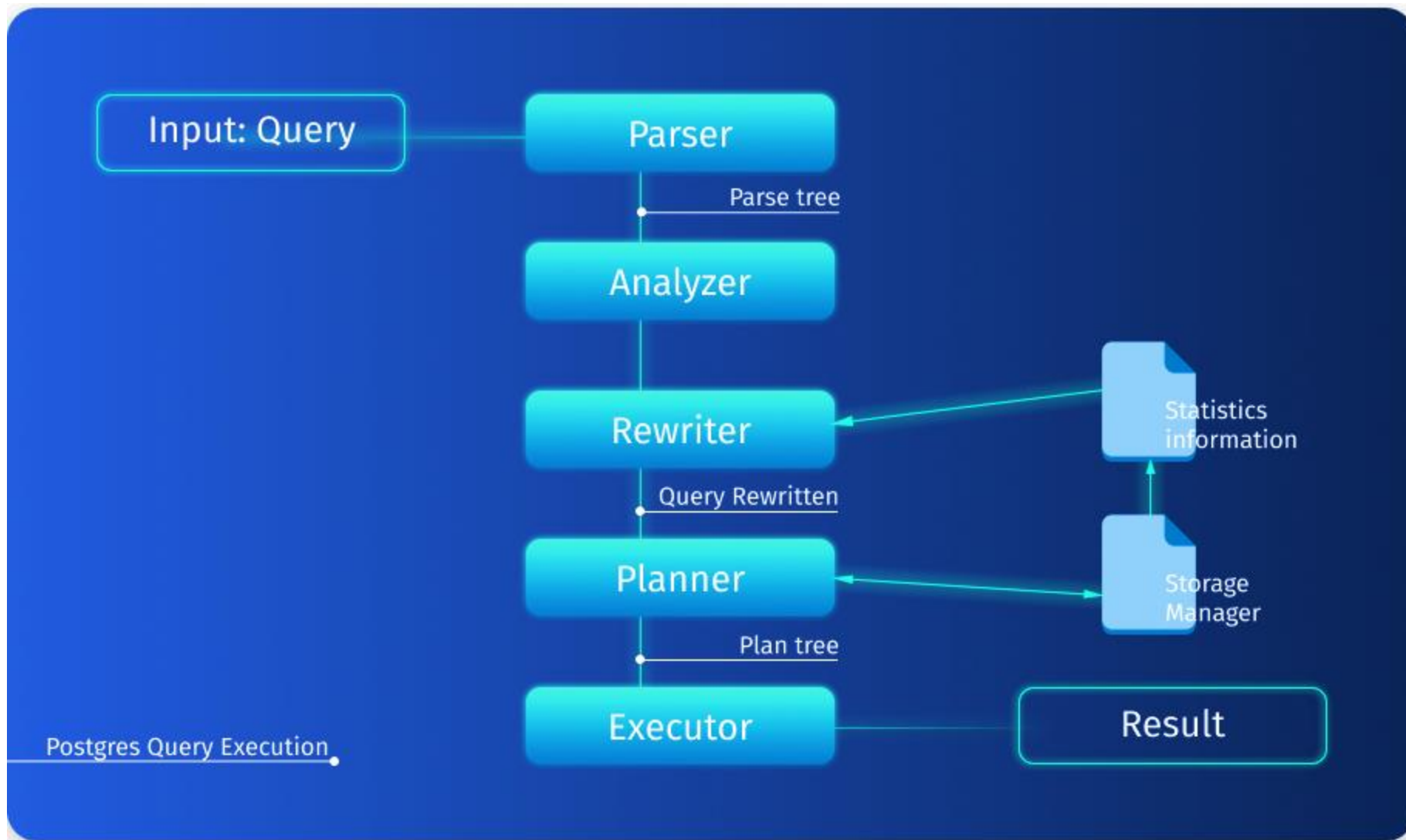
```
-----  
Seq Scan on employee (cost=0.00..4.12 rows=4 width=34)  
Filter: (designation_id = 103)  
(2 rows)
```

Type of scan

*Cost of the operation to retrieve
rows from the employee table*

Estimated number of rows

PostgreSQL QUERY EXECUTION



KEY COMPONENTS OF THE EXECUTION PLAN

- Node Type:
 - This indicates the type of operation being performed, such as Seq Scan, Index Scan, Hash Join, etc.
- Cost Estimates:
 - Startup Cost: This is the cost to start returning rows from the operation.
 - Total Cost: This is the estimated total cost of executing the operation and returning all rows.
- Row Estimates:
 - This shows the estimated number of rows that will be processed or returned by that operation in the execution plan.

KEY COMPONENTS OF THE EXECUTION PLAN

- Width:
 - The average width (in bytes) of the rows being processed or returned by the operation.
- Filter Conditions:
 - This indicates any filtering criteria that will be applied during the execution, such as those specified in a WHERE clause.
- Join Types:
 - If there are join operations, this part indicates the methods used (e.g., Nested Loop, Hash Join).

TYPES OF TABLE SCANS

- In PostgreSQL, there are several types of table scans that the query planner can utilize when executing SQL queries.
- **Sequential Scan (Seq Scan)**
 - A sequential scan reads through all rows in a table one by one, checking each row against the query conditions.
- **Index Scan**
 - An index scan uses an index to directly access the rows that meet the query conditions.
 - It retrieves rows by looking up index entries rather than scanning the entire table.

There are more types of tables scans such as Bitmap Heap Scan, Partial Index Scan etc.

TYPES OF TABLE SCANS

- **Bitmap Scan:**
 - This method combines multiple index scans. It creates a bitmap of the rows that match the criteria. Then, it retrieves the actual rows in a single pass. This is efficient for complex queries.

There are more types of tables scans such as Bitmap Heap Scan, Partial Index Scan etc.

Physical Database

WHAT IS A PHYSICAL DATABASE ?

- A physical database refers to the actual storage of data on physical hardware.
- It includes how data is organized, stored, and accessed on disk in terms of files and records.
- The physical database is concerned with how this data is physically laid out on storage devices.

KEY ASPECTS OF A PHYSICAL DATABASE

- **File Structures:** Data is stored in files on disk, and these files can be organized in different ways:
 - **Heap Files:** Unordered files where records are placed wherever there is free space.
 - **Ordered Files:** Files where records are sorted based on certain attributes, such as primary key values.
- **Access Methods:** The physical database determines how quickly data can be accessed and retrieved, depending on how it is physically organized and indexed.

KEY ASPECTS OF A PHYSICAL DATABASE

- **Indexes:**
 - Indexes (like primary, secondary, and clustered) help the database locate data more efficiently by creating references to where the data is stored physically.
- **Blocks and Pages:**
 - The database stores data in blocks or pages, which are fixed-size chunks that the system reads from or writes to disk.
 - The way these blocks are organized impacts performance.

File Structure

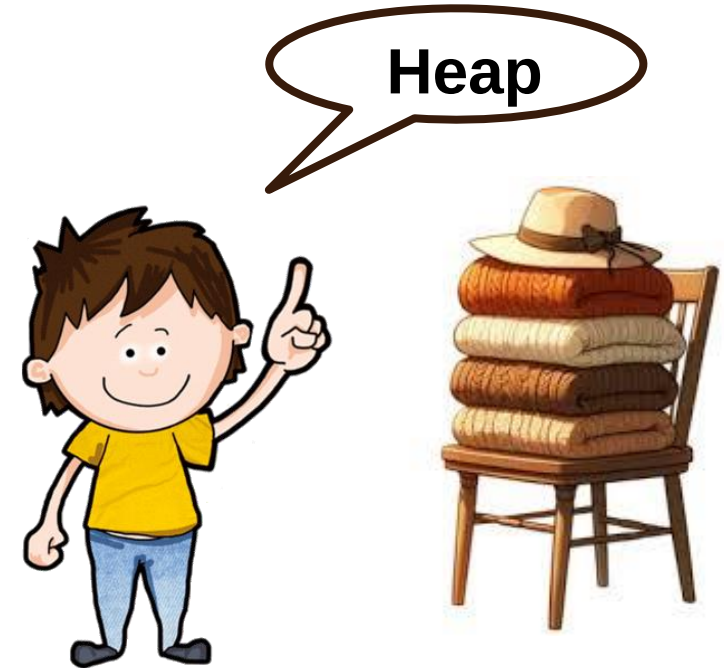
FILE STRUCTURE

- A physical database either be stored in an ordered file or in an unordered file(heap).
- Both approaches have their own advantages and disadvantages.

	Ordered	Unordered (Heap)
INSERT	Difficult	Easy
SEARCH	Easy	Difficult



HEAP VS. ORDERED FILES



HEAP VS. ORDERED FILES

Heap	Ordered Files
• An unsorted collection of records, where new records are inserted wherever free space is available in the file.	• Records are stored in a specific order based on the value of one or more attributes (typically the primary key or some other index key).
• No specific ordering of records is maintained	• Data is physically organized in a sorted sequence.

Physical Database Tuning

WHAT IS PHYSICAL DATABASE TUNING?

- Physical Database Tuning is the process of optimizing how data is stored and accessed to improve database performance.

It involves:

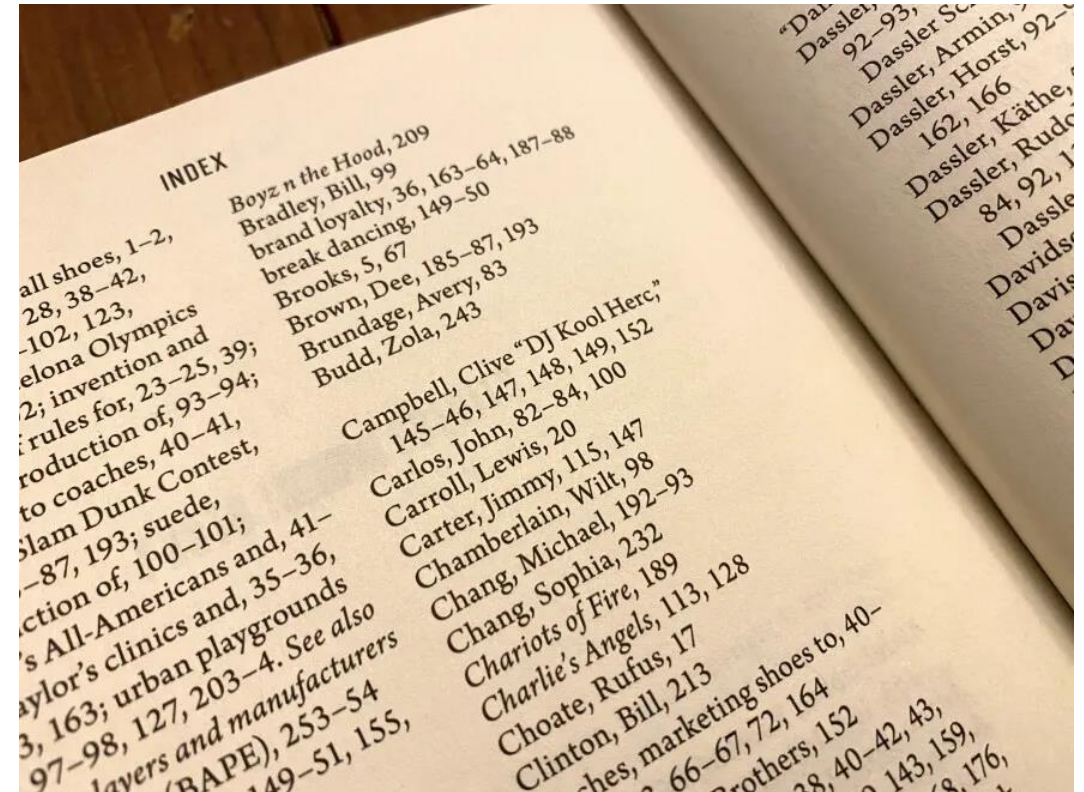
- **Disk Storage:** Efficiently managing data files.
- **Indexing:** Creating indexes to speed up searches.
- **Partitioning:** Splitting large tables into smaller parts.
- **Caching:** Storing frequently used data in memory.
- **I/O Optimization:** Reducing disk reads and writes for faster access.

The goal is to make the database faster and more efficient by focusing on physical resources..

Indexing

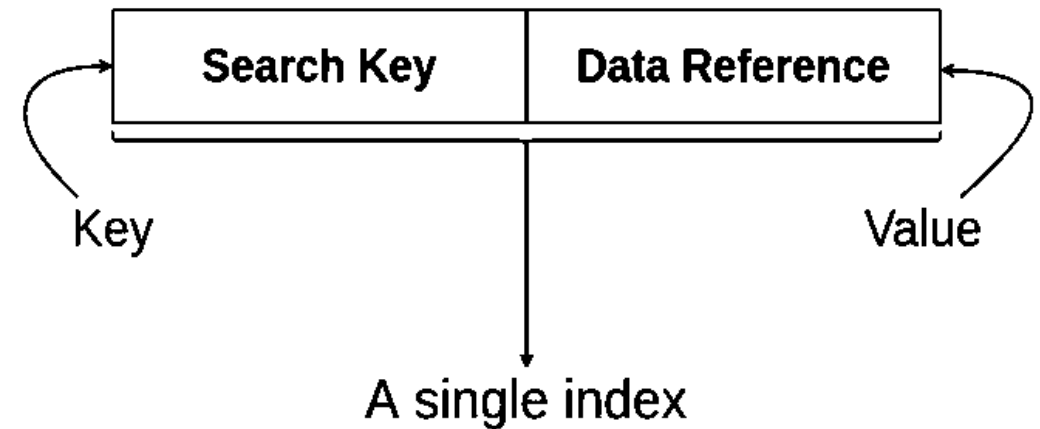
WHAT IS INDEXING?

- A technique used in databases to optimize query performance by minimizing the number of disk accesses required when a query is processed.
- The index is a type of data structure.
- It is used to locate and access the data in a database table quickly.



STRUCTURE OF AN INDEX IN DATABASE

- **Search key (Indexing Column)** - Contains a copy of the primary key or candidate key of the table.
- The values of the primary key are stored in sorted order so that the corresponding data can be accessed easily.
- **Data Reference** - Contains a set of pointers holding the address of the disk block where the value of the particular key can be found.



HOW INDEX WORKS?

Database Index

Search Key	Pointer to Disk Location
Alice	149
David	2459
Greg	3368

149

Employee_Name	Employee_ID	Department
Alice	43	Marketing
Anna	504	Finance

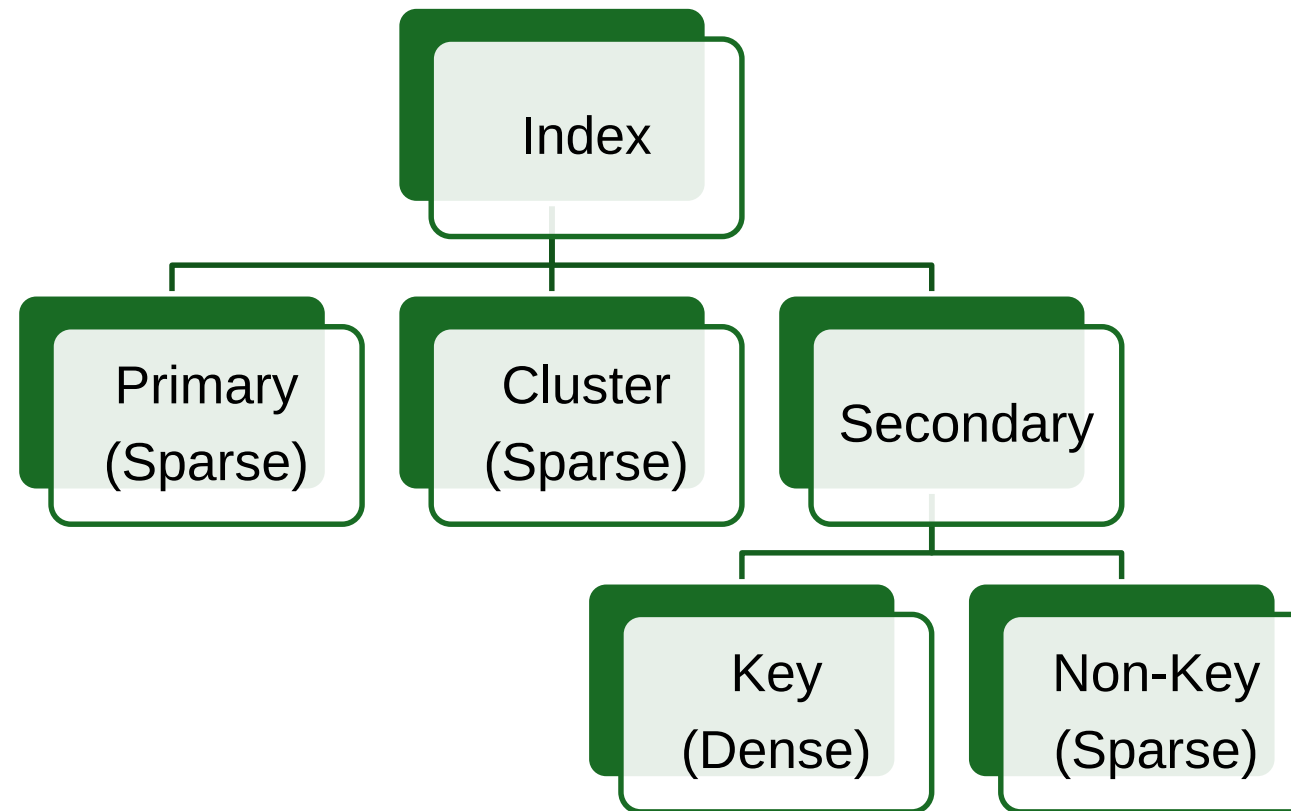
2459

David	342	Finance
Derek	57	Sales
Danielle	245	Sales

3368

Greg	134	Operations
Gabby	187	Marketing

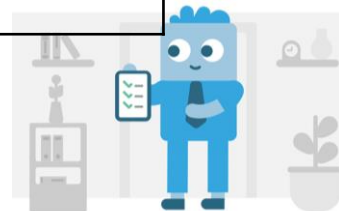
TYPES OF INDEXES



PRIMARY INDEX VS. CLUSTER INDEX VS. SECONDARY INDEX

Index Type	Nature of Relation	Nature of the indexing field	Number of tuples in the index	Density
PRIMARY Ordered with emp_id	Physically ordered by the indexing field	Key (candidate key) key	Number of blocks of the relation	Non-dense (Sparse)
CLUSTER Ordered with gender & indexing with gender	Physically ordered by the indexing field	Non – key (foreign key) Not a key	Number of distinct index field values ②	Non-dense (Sparse)
SECONDARY Ordered with designation & indexing with gender	Either an unordered relation or physically ordered by a field other than the indexing field.	Key or non key Non key	Number of records or number of distinct index field values Two distinct values - ②	<u>Dense</u> or Non-dense ↓ Secondary non-key

Dense – indexing that includes an entity for every record



Drop Index

Once you identify an unused index, you can drop it with the following command:

```
DROP INDEX index_name;
```

Best Practices

- Review Periodically: Regularly check for unused indexes, especially after significant changes in application logic or database schema.
- Analyze Query Performance: Consider the overall query performance impact of dropping an index, as an index may be planned for future use or used infrequently.
- Document Changes: Keep a log of indexes added or removed for auditing and future reference.

Primary Index

PRIMARY INDEX

- Index created on the primary key column(s) of a table.

Characteristics

- Unique for each record.
- Automatically created in most DBMS when a primary key is defined.

Example

- If a table has a StudentID as the primary key, the primary index will store StudentID values in sorted order for fast access.

Partial Index

- Partial indexes allow you to index only a subset of rows in a table based on a specified condition.

A partial index is created with a condition that limits the rows included in the index.

ACTIVITY 3

- View the indexes in the employee table.

```
\d employee;
```

- Drop the primary index from the table.

```
ALTER TABLE employee DROP INDEX employee_pkey;
```

- Retrieve the cost for selecting a particular employee.

```
EXPLAIN SELECT * FROM employee WHERE id = 25;
```

- Designate id as the primary key for the employee table. This will automatically create a primary index for the employee table.

```
ALTER TABLE employee ADD PRIMARY KEY (id);
```

- Retrieve the cost for selecting a particular employee.

```
EXPLAIN SELECT * FROM employee WHERE id = 25;
```

- Observe that the second query uses the primary index resulting a low cost.

DESIGNATION RELATION

Designation	
Designation_id (20 B)	Name (60 B)
101	Lecturer
102	Clerk
103	Manager
107	Demonstrator
109	Researcher
251	Registrar
259	Assistant Registrar
261	Security Officer

Block size – 256 Bytes.
The block pointer is 4 Bytes long.

251 distinct designations

Block 1 (Designation)

Block 2 (Designation)

Block 83 (Designation)

Block 84 (Designation)

TUPLE SIZE (DESIGNATION)

$\lfloor 5.6 \rfloor$ = Floor Function = 5

$\lceil 3.2 \rceil$ = Ceiling Function = 4

$$\begin{aligned}\text{Tuple size (Designation)} &= \sum \text{Attribute size} \\ &= 20 + 60 = 80 B\end{aligned}$$

Block Factor (Designation)

$$\begin{aligned}\text{Block Factor (Designation)} &= \left\lfloor \frac{\text{Block Size}}{\text{Tuple Size(Designation)}} \right\rfloor \\ &= \left\lfloor \frac{256 \text{ B}}{80 \text{ B}} \right\rfloor \\ &= \lfloor 3.2 \rfloor \\ &= 3\end{aligned}$$

NUMBER OF BLOCKS (DESIGNATION)

$$\begin{aligned}\text{Number of Block}(\textit{Designation}) &= \left\lceil \frac{\textit{Number of Tuples}(\textit{Designation})}{\textit{Blocking Factor}(\textit{Designation})} \right\rceil \\ &= \left\lceil \frac{251}{3} \right\rceil \\ &= \lceil 83.66 \rceil \\ &= 84\end{aligned}$$

COST OF A QUERY (BEFORE PRIMARY INDEX)

`SELECT name from Designation WHERE id=25;`

(Designation Relation is physically ordered by the field designation_id therefore, perform binary search)

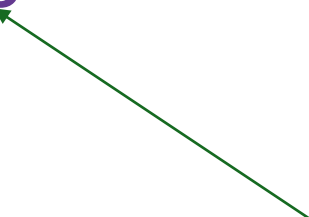
Designation Relation is physically ordered by the field designation_id

$$\text{Cost} = \lceil \log_2(\text{Number Of Blocks}) \rceil = \lceil \log_2 84 \rceil = 6.39 \text{ IO}$$

Binary Search



Input Output Operations



TUPLE SIZE (PRIMARY INDEX ON DESIGNATION)

- We can reduce the cost by introducing a primary index on “ Designation Relations”.

Primary Index on “Designation Relations”.

Tuple Size(Primary Index Of Designation)

$$\sum \text{Attribute Size} = 20 + 4 = 24 \text{ B}$$

Block Pointer Size (Given)

BLOCKING FACTOR (PRIMARY INDEX ON DESIGNATION)

$$\begin{aligned}\text{Blocking Factor (Primary Index Of Designation)} &= \left\lfloor \frac{\text{Block Size}}{\text{Tuple Size (Primary Index of Designation)}} \right\rfloor \\ &= \left\lfloor \frac{256 B}{24 B} \right\rfloor = \lfloor 10.67 \rfloor \\ &= 10\end{aligned}$$

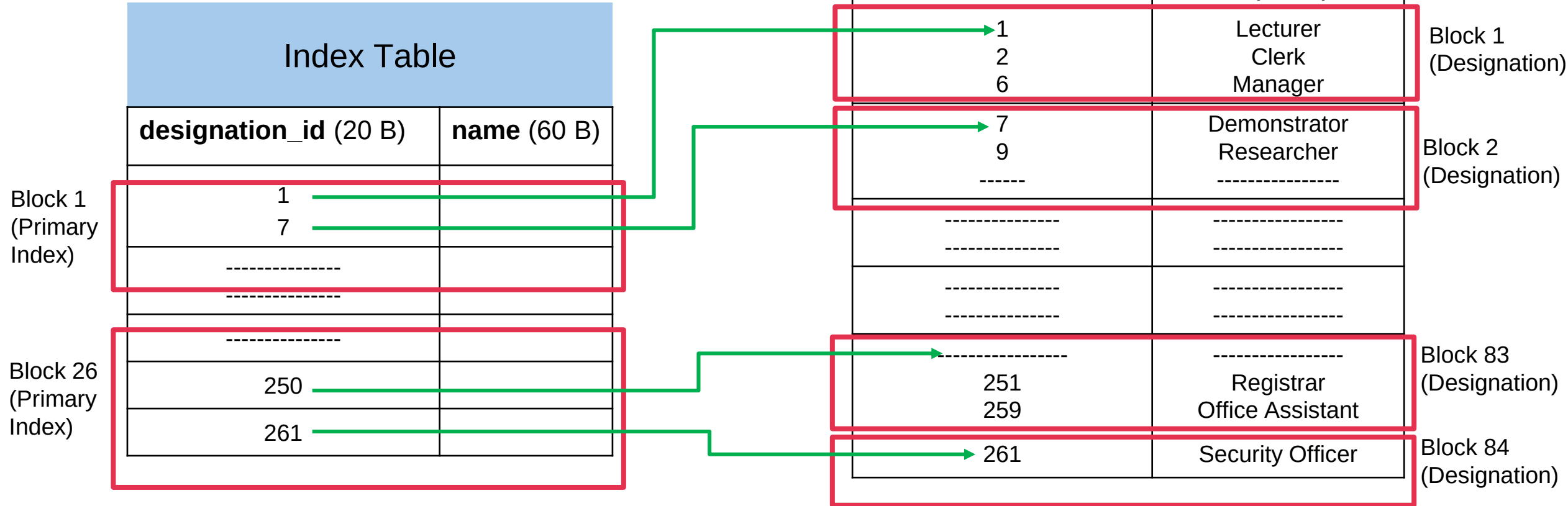
This means only 10 tuples are in a block.

$$\begin{aligned}\text{No of Blocks (Primary Index Of Designation)} &= \left\lceil \frac{\text{Number of Tuples}}{\text{Blocking Factor}} \right\rceil \\ &= \left\lceil \frac{251}{10} \right\rceil \\ &= \lceil 25.1 \rceil \\ &= 26\end{aligned}$$

No of tuples in The Designation Relation

CONCEPTUAL VIEW (PRIMARY INDEX OF DESIGNATION)

Therefore the “Primary Index of Destination” would look as below



Index does not have a user or external view.

COST OF A QUERY (AFTER PRIMARY INDEX)

When querying a database, if an index is present, the database engine can use this formula to estimate how many I/O operations will be needed to retrieve data.

$$\text{Cost} = \lfloor \log_2(\text{No of Blocks}) \rfloor + 1 = \lfloor \log_2 26 \rfloor + 1 = \lfloor 4.07 \rfloor + 1 = 4 + 1 = 5 \text{ IO} < 6.39 \text{ IO}$$

It read for Index

(Main table IO)

To get the Value of the identified block

Cost of a query (before primary index)

QUESTION

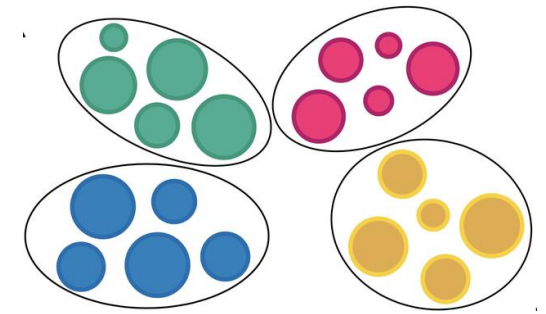
Write a query execution plan to find the designation with ID=251 with Order of Primary Index.

Find the lesser closer value or equal to 251 in the index table. Then find the selected block in the designation table using the pointer. The cost of that process is 4 IO. Then the value is in the block and we need another 1 IO to find value 251 from the selected block.

Cluster Index

CLUSTERING INDEX

- Orders the actual data in the table according to the index.
- **Characteristics:**
 - Only one clustering index per table.
 - Makes range queries faster.
- ***Example***
- If you frequently query a range of values (like dates), a clustering index on the DateOfBirth column can improve performance.



CLUSTERING INDEX (NON-KEY, ORDERED)

- Observe that the “employee Relation” is physically ordered by the field “designation_id”.
- Since “designation_id” is not a key field. We can not create a primary index.
- Instead, let’s design a “Cluster index on employee”.
- Employee table has 100000 rows.
- Column sizes in employee Relation are as follows:
employee_id = 20 B, name = 68 B, gender_id = 20 B, designation_id = 20 B

$$\begin{aligned}\text{Tuple Size (employee)} &= \sum \text{Attribute Size} \\ &= \text{employee_id} + \text{name} + \text{gender_id} + \text{designation_id} \\ &= 20 + 68 + 20 + 20 = 128 \text{ B}\end{aligned}$$

$$\text{Blocking Factor (employee)} = \left\lfloor \frac{\text{Block Size}}{\text{Tuple Size (Employee)}} \right\rfloor = \left\lfloor \frac{256 \text{ B}}{128 \text{ B}} \right\rfloor = \lfloor 2 \rfloor = 2$$

This means only 2 tuples of “Employee Relation” can be stored in a block.

NUMBER OF BLOCKS (EMPLOYEE)

$$\begin{aligned}\text{No of Blocks (Employee)} &= \left\lceil \frac{\text{Number of tuples}(\text{Employee})}{\text{Blocking Factor}(\text{Employee})} \right\rceil \\ &= \left\lceil \frac{10\ 0000}{2} \right\rceil \\ &= [50\ 000] \\ &= 50\ 000\end{aligned}$$

EMPLOYEE TABLE

employee_id (20B)	name (68B)	gender_id (20B)	designation_id (20B)
1	Janaka	1	1
2	Saman	1	1
3	Kamal	1	1
5	Saman	2	1
4	Lal	1	200
2000	Kamal	1	200
100	Sarath	1	259
9800	Namal	1	259

Block 1

Block 2

Block 49999

Block 50000

COST OF A QUERY (BEFORE CLUSTER INDEX)

Consider the query,

```
SELECT name FROM Employee WHERE Designation_id = 251 ;
```

Since 'Employee Relation is physically Ordered by the field 'Designation_id' (clustered), we can perform a binary search on it.

$$\begin{aligned}\text{Cost} &= \text{Height of the tree} &&= \lfloor \log_2 (\text{Number of Blocks in (Employee)}) \rfloor \\ &&&= \lfloor \log_2 (50\,000) \rfloor \\ &&&= \lfloor 15.61 \rfloor \\ &&&= 15 \text{ IO}\end{aligned}$$

CLUSTER INDEX ON DESIGNATION

- Let's see how we can reduce costs by introducing a cluster index on 'employee Relation'.

Tuple Size (Cluster Index of Employee) = $\sum \text{Attribute Size}$

$$\begin{aligned} &= \text{Designation_id column size} + \text{Pointer column size} \\ &= 20 + 4 = 24 \text{ B} \end{aligned}$$

Cluster Index of Designation	
Designation_id (20 bytes)	Pointer (4 bytes)
1	
2	
259	
261	

Block 1
(Index)

Block 25
(Index)

Blocking Factor(Cluster Index of Employee)

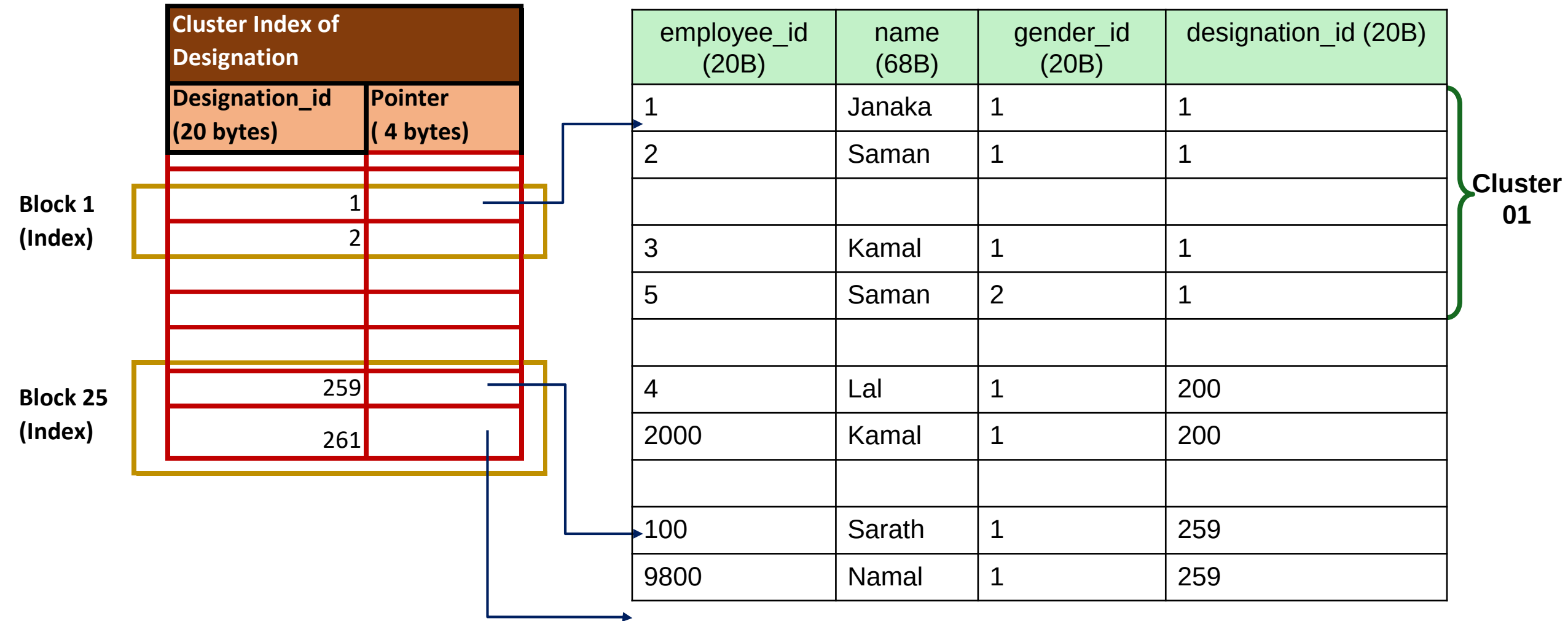
$$\begin{aligned} &= \left\lfloor \frac{\text{Block Size}}{\text{Tuple Size}(\text{Cluster Index of Employee})} \right\rfloor \\ &= \left\lfloor \frac{256 \text{ B}}{24 \text{ B}} \right\rfloor = \lfloor 10.67 \rfloor = 10 \end{aligned}$$

This means only 10 tuples of the " Cluster Index of Employee" can be stored in a block.

NUMBER OF BLOCKS – CLUSTER INDEX OF DESIGNATION

$$\begin{aligned}\text{No of blocks (Cluster Index of Employee)} &= \left\lceil \frac{\text{Number of Tuple(Cluster Index Of Employee)}}{\text{Blocking Factor(Cluster Index of Employee)}} \right\rceil \\ &= \left\lceil \frac{\text{Distinct values in the Designation Table}}{\text{Blocking Factor (Cluster Index of Employee)}} \right\rceil \\ &= \left\lceil \frac{251}{10} \right\rceil = \lceil 25.1 \rceil \\ &= 26\end{aligned}$$

Conceptual View - Cluster Index of Designation



COST OF A QUERY – (AFTER CLUSTER INDEX)

Reconsider the query

```
SELECT name FROM Employee WHERE Designation_id = 259 ;
```

- Since the index file has been ordered, we can first perform a binary search on the 'cluster Index of employee' to find the pointer for the 'employee Relation'.
- Then we need one more block read on the 'Employee Relation to find the needed tuple.

$$\begin{aligned}\text{Cost} &= \text{Height of the index tree} + \mathbf{1} \\ &= \lfloor \log_2(\text{No of Blocks (Cluster Index)}) \rfloor + 1 \quad \leftarrow \text{Best Case} \\ &= \lfloor \log_2 26 \rfloor \\ &= \lfloor 4.7 \rfloor + 1 = 4 + 1 = 5 \text{ IO} < 15 \text{ IO}\end{aligned}$$

PRIMARY INDEX VS CLUSTER INDEX

Aspect	Primary Index	Clustered Index
Purpose	Ensures data uniqueness and retrieval via primary key	Orders data physically for optimized searches
Uniqueness	Always unique	Can be unique or non-unique
Data Storage	Separate from actual data unless used as a clustered index	Data stored in the index structure
Quantity	Only one primary index per table	Only one clustered index per table
Physical Order	Does not define physical data order unless it's also clustered	Defines the physical data order

ACTIVITY 4 – CLUSTER INDEX

Activity



1. Use the employee table created before.
2. Create an index on designation_id of the employee table.

```
CREATE INDEX employee_designation_idx  
ON employee (designation_id);
```
3. Cluster Employee table by that index created above.

```
CLUSTER employee USING employee_designation_idx;
```
4. Then display all the data in the Employee table including ctid.

```
SELECT *, ctid FROM employee;
```
5. Insert a new record to the Employee table.

```
INSERT INTO employee (name, nic, designation_id) VALUES ('Alice',  
                '1000008888V', 1)
```
6. Then display all the data in the Employee table including ctid.
7. Repeat the step 4.
8. Run the command in step 3 again.
9. Repeat step 4.

Past Papers

Oct 2024/Question 1



Question 1

[Total – 20 marks]

Consider the Medicine table given below. Its primary key 'medicine_id' while the 'category_id' is a foreign key referencing the primary key of the Category table.

Medicine		
medicine_id (20 B)	name (64 B)	category_id (20 B)

This table is physically ordered by the 'category_id'. It contains **675** tuples across **43** distinct categories. Note that it is stored on a disk with **256 B** blocks and **4 B** block pointers.

Past Papers

Oct 2024/Question 1



(a) Name **two (2)** pieces of information that can be obtained by executing the following query.

```
EXPLAIN SELECT * FROM Medicine WHERE category_id = 'M098';
```

[2.0 marks]

(b) Determine whether each of the following combinations of indexes can **exist** on the Medicine Table.

- i. A primary index and a cluster index
- ii. A primary index and a secondary index
- iii. Multiple cluster indexes

[1.0 × 3 = 3.0 marks]

(c) Calculate each of the following estimates on the '**Medicine**' table.

- i. Tuple size
- ii. Blocking Factor
- iii. Number of Blocks

[2.0 × 3 = 6.0 marks]

Past Papers

Oct 2024/Question 1



(d) Calculate each of the following estimates on the **cluster index** on 'category_id' of the 'Medicine' table.

- i. Tuple size [2.0 marks]
- ii. Blocking Factor [1.0 marks]
- iii. Number of Blocks [2.0 marks]

(e) Answer the following questions, assuming that the tuples in the Medicine table are uniformly distributed across all categories (i.e., each category contains the same number of medicine).

- i. How many **blocks** of 'Medicine' table belongs to each category?
- ii. Hence, show that the **cost** of the query, "SELECT * FROM Medicine WHERE category_id = 'M098' ;" can be reduced to **9 IO**. [2.0 × 2 = 4.0 marks]

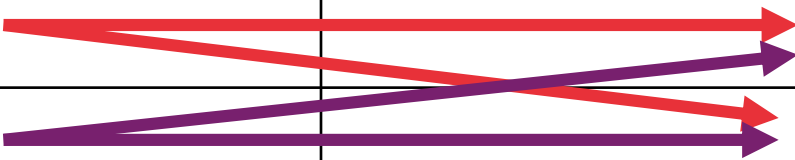
Secondary Index

SECONDARY INDEX

- Index created on a column that is not the primary key.
- Helpful for searching columns that are frequently queried but not the primary key.
- *Example*
- An index on the LastName column in a student database

SECONDARY INDEX CONTD..

Table Already Sorted By a Different Column	Nature of the Secondary Index
Key (In case of Primary Index)	Key
Non-Key (In case of Cluster Index)	Non-Key



1. Primary Index + Key Secondary
2. Primary Index + Non-key Secondary
3. Cluster Index + Key Secondary
4. Cluster Index + Non-key Secondary

1. PRIMARY INDEX + KEY SECONDARY

Index Table

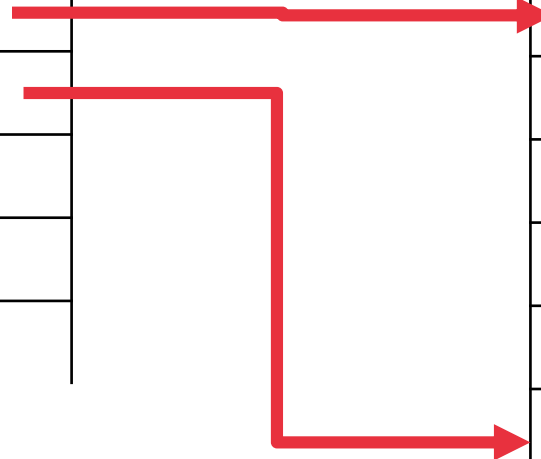
Nic	Pointer
921102226V	
901234888V	
.	
.	
.	

251 rows

Designation Table

Id	Name	Nic	Department_id
1	HoD	921102226V	2
2	Manager	992764782V	2
.	.	.	.
.	.	.	.
.	.	.	.
251	Secretary	901234888V	1
258	Registrar	984442328V	2

251 rows



2. PRIMARY INDEX + NON-KEY SECONDARY

Deaprtment Table

Department_id	Name
1	Computing
2	Admission

Index Table

Dept_id	Pointer
1	
2	
2	

Designation Table

Id	Name	Nic	Department_id
1	HoD	921102226V	2
2	Manager	992764782V	2
.	.	.	.
.	.	.	.
.	.	.	.
251	Secretary	901234888V	1
258	Registrar	984442328V	2

251 rows

251 rows

3. CLUSTER INDEX + KEY SECONDARY

Index Table

Id	Pointer
1	
2	
3	
5	
6	

251 rows

Employee Table

Id	Name	Gender_id
1	Nayana	1
3	Leela	1
2	Saman	2
5	Nimal	2
6	Kamal	2

251 rows

4. CLUSTER INDEX + NON-KEY SECONDARY

Index Table

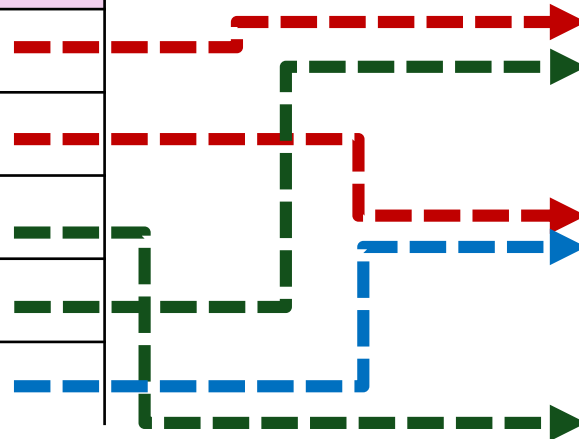
Desig_id	Pointer
1	
1	
2	
2	
3	

251 rows

Employee Table

Id	Name	Gender_id	Desig_id
1	Nayana	1	2
3	Leela	1	1
2	Saman	2	3
5	Nimal	2	1
6	Kamal	2	2

251 rows



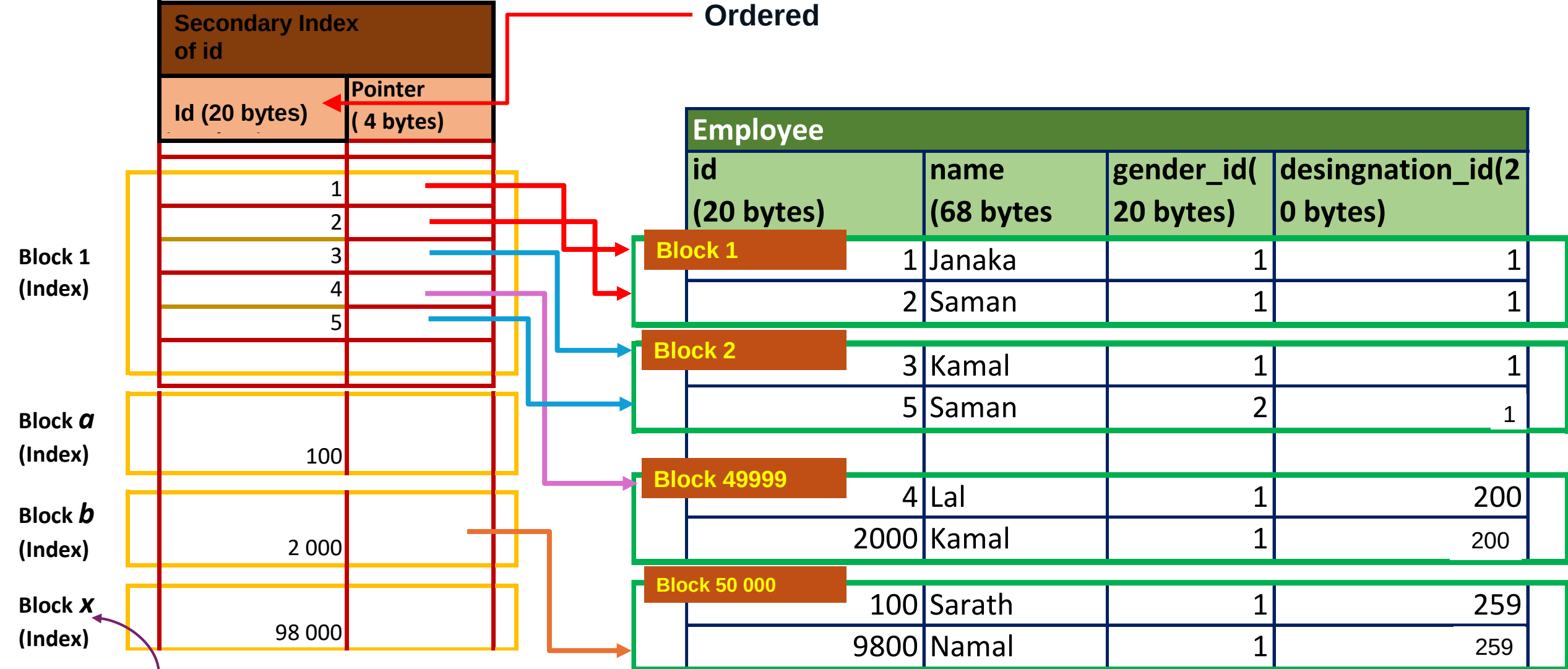
SECONDARY INDEX-KEY

- Consider the query,

```
SELECT name FROM Employee WHERE id = 3 ;
```

- Observe that Employee table (in slide 83) is not ordered by the field id (**Ordered by designation_id**).
- Therefore, we can not create either a primary index or cluster index Employee based on 'id'.
- Id is the candidate key here.
- Case: 3 (Cluster index + key Secondary)

SECONDARY INDEX-KEY



Can not find the Block number here.
Few data in the employee table are given here.

COST OF A QUERY (BEFORE SECONDARY INDEX)

Calculate the cost of the query.

```
SELECT name FROM Employee WHERE id = 3 ;
```

Since this table is not ordered we use linear Search.

Cost = Number of blocks (Employee) = 50 000 IO

TUPLE SIZE, BLOCKING FACTOR - SECONDARY INDEX

Tuple Size (Secondary Index of Employee) = $\sum$ Attribute Size = id size + pointer size = 20 + 4 = 24 Bytes

$$\begin{aligned}\text{Blocking Factor(Secondary Index of Employee)} &= \left\lfloor \frac{\text{Block Size}}{\text{Tuple Size(Secondary Index of Employee)}} \right\rfloor \\ &= \left\lfloor \frac{256 \text{ Bytes}}{24 \text{ Bytes}} \right\rfloor \\ &= \lfloor 10.67 \rfloor \\ &= 10\end{aligned}$$

This means only 10 tuples of the 'Secondary Index of the Employee' can be in a block.

Number of Blocks (Secondary Index of Employee(Designation id))

$$\text{No of Blocks(Secondary Index of Employee)} = \left\lceil \frac{\text{Number of tuples} \times (\text{Secondary Index of Employees})}{(\text{Blocking Factor} \times (\text{Secondary Index of Employee}))} \right\rceil$$

This is **Secondary** key therefore number of records in tuple size.

$$= \left\lceil \frac{\text{No of tuples in Employee table}}{\text{Blocking factor}} \right\rceil$$

$$= \left\lceil \frac{10\ 0000}{10} \right\rceil = \lceil 10\ 000 \rceil = 10\ 000$$

Therefore the “*Secondary Index of Employees*” would like above.

LET'S CALCULATE COST WHEN HAVING ORDERED INDEX.....

$$\text{Cost} = \lfloor \log_2 (\text{Number of Blocks}) \rfloor + 1$$

$$= \lfloor \log_2 (10000) \rfloor + 1$$

$$= \lfloor 13.29 \rfloor + 1$$

$$= 13 + 1$$

$$= 14 \text{ IO} < 50000 \text{ IO}$$

SECONDARY INDEX – NON KEY

Reconsider the query,

```
SELECT name FROM Employee WHERE gender_id = 1 ;
```

1. Employee table is not ordered by 'gender_id'. (*Because it is ordered by 'designation_id'.*)
2. Therefore, we can't create either a primary index or clustered index.
3. Therefore, we design a secondary index.
4. Gender_id is a non-key. Therefore, we have to go for a Secondary (non key)index.

Tuple Size, Blocking Factor (Secondary Index of Employee(Gender_id))

Tuple size (*Secondary Index of Employee*)

$$= \sum \text{Attribute size} = 20 + 4 = 24 \text{ bytes}$$

Blocking Factor(*Secondary Index of Employee*)

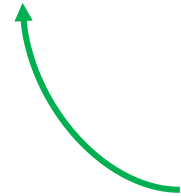
$$= \left\lfloor \frac{\text{Block Size}}{\text{Tuple Size}} \right\rfloor$$

$$= \left\lfloor \frac{256 \text{ B}}{24 \text{ B}} \right\rfloor$$

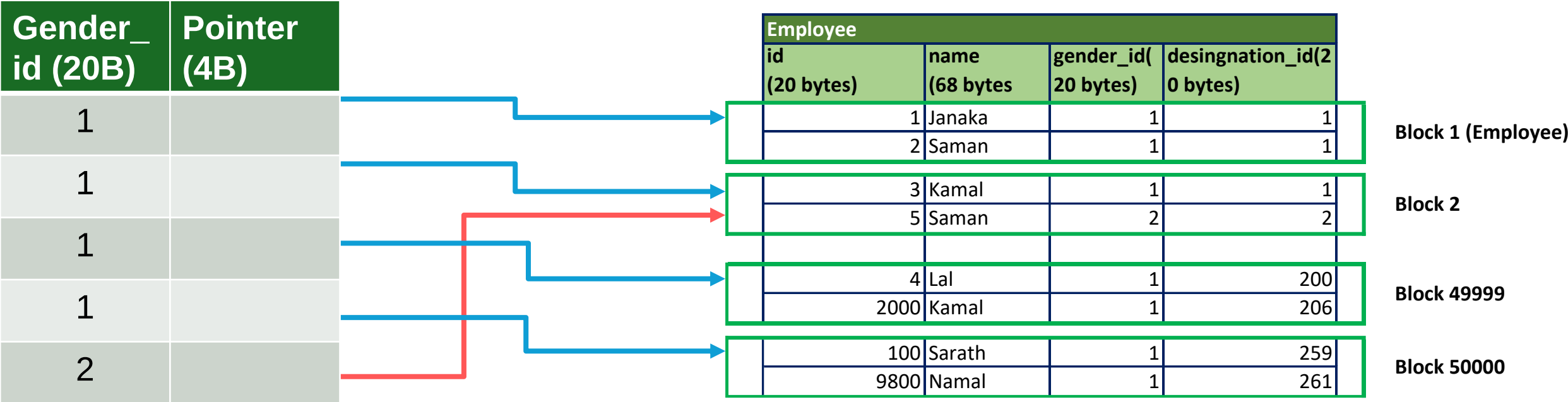
$$= \lfloor 10.67 \rfloor$$

$$= 10$$

Gender_id size



CONCEPTUAL VIEW – SECONDARY INDEX OF EMPLOYEE



The actual tuple count in the index depends on how it's structured and the distribution of gender_id values, not simply on the total rows in the employee table.

ACTIVITY 5 – PARTIAL INDEX

Activity



1. Create a partial index on the employee table for entries where the designation is equal to 3 (or any).

```
CREATE INDEX index_designation_3 ON employee (designation_id) WHERE designation = 3;
```

2. Check the number of rows in the Partial Index.

```
SELECT COUNT(*) FROM employee WHERE designation_id = 3;
```

3. Explain query using the Partial Index.

```
EXPLAIN SELECT * FROM employee WHERE designation_id = 3 AND gender = 2;
```

4. Explain query that does not use the Partial Index.

```
EXPLAIN SELECT * FROM employee WHERE designation_id = 2 AND gender = 2;
```

STATISTICS FOR INDEXES

- Using pg_stat_user_indexes:
- PostgreSQL tracks usage statistics for indexes.
- You can query the pg_stat_user_indexes and pg_index tables to find indexes that have not been used since the last reset.

```
SELECT
    idx.indrelid::regclass AS table_name,
    idx.indexrelid::regclass AS index_name,
    stat.idx_scan
FROM
    pg_index AS idx
JOIN
    pg_stat_user_indexes AS stat ON idx.indexrelid = stat.indexrelid
WHERE
    idx.indisvalid AND stat.idx_scan = 0
ORDER BY
    table_name, index_name;
```

Index Only Scan

Index Only Scan

- An Index Only Scan in PostgreSQL is a fast way to get data directly from the index, without needing to access the actual table.
- This works when the query can be fully answered using just the information stored in the index, including both the indexed columns and any extra columns included in the index (through a covering index).
- To answer certain queries, it is sufficient to scan only the index.

EXAMPLE

Ex 1:

SELECT DISTINCT designation_id FROM employee;

- Suppose we have a cluster index on employee based on designation_id.
- Therefore, in this case we can use an index only scan.

Ex 2:

Suppose NIC in employee table is unique.

- If we have a secondary key index based on NIC on employee table “an index only scan” can be used to check the uniqueness before insertion and update.

Activity 6 - In insert operation

Use the previous 'employee' table.

1. Run a query that searches by 'nic' only, and check the cost with EXPLAIN.

```
EXPLAIN ANALYZE SELECT * FROM employee WHERE nic = '100000165V';
```

2. Create an index on 'nic'.

```
CREATE INDEX idx_employee_nic ON employee (nic);
```

3. Run the same query again and compare the cost.

```
EXPLAIN ANALYZE SELECT * FROM employee WHERE nic = ' 100000165V ';
```

4. Using the index to check if a nic exists before INSERT.

```
EXPLAIN SELECT 1 FROM employee WHERE nic = ' 100000165V';
```

ACTIVITY 7 - INDEX ONLY SCAN

Activity



1. Create employee table as below:

```
employee(employee_id, name, gender_id, designation_id);  
(employee_id – primary key)
```

2. Add data to the Employee table;
3. Explain the following query;

```
SELECT * FROM employee WHERE employee_id BETWEEN 50 AND 55;
```

4. Observe the scan that is used to execute the query.
5. Explain the following query;

```
SELECT COUNT(id) FROM employee WHERE employee_id BETWEEN 50 AND 55;
```

6. Observe the scan that is used to execute the query.

Note: To use auto increment use, GENERATED ALWAYS AS IDENTITY.

Unused Index

UNUSED INDEXES

- Unused indexes in databases are those that exist but are not actively used by queries to retrieve or sort data.
- Over time, these indexes can accumulate due to schema changes, new requirements, or adjustments in application logic, leading to performance issues.

IMPLICATIONS OF UNUSED INDEXES

- **Increased Storage Costs:**
 - Unused indexes occupy disk space, which can be significant in large databases with many columns and complex data structures.
- **Slower Data Modification:**
 - Indexes need to be updated with each data modification (insert, update, delete).
 - If indexes are unused, this maintenance becomes unnecessary overhead, slowing down data modification operations.
- **Memory and Cache Consumption:**
 - Unused indexes can take up valuable cache space, reducing the availability of memory resources for more frequently used indexes and other critical database operations.

ACTIVITY – UNUSED INDEXES

Activity



- Student (**RegNo**, IndexNo, Name, Gender_ID)
- Create two indexes in same column.
- Execute queries based on the created indexes.
- Use **pg_stat_user_indexes** to check which is the unused index.

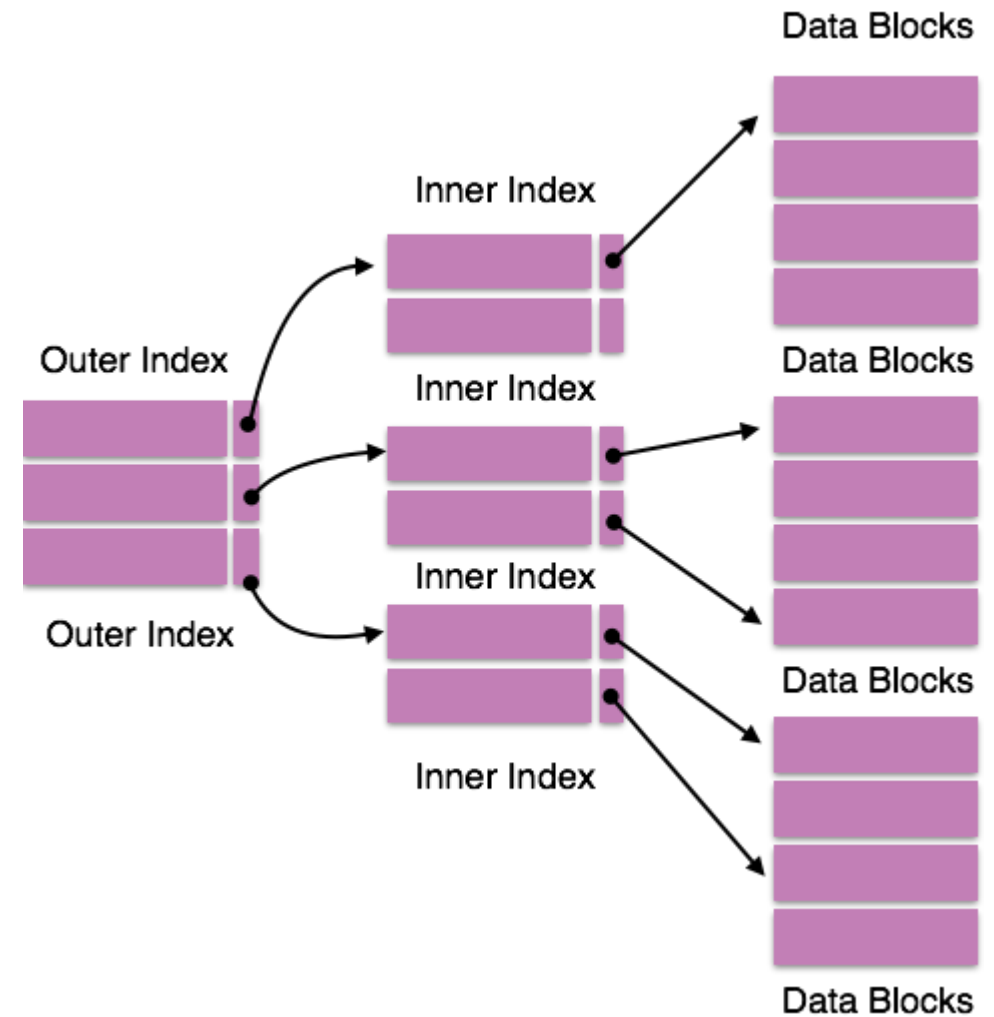
Multi-Level Indexing

NEED OF MULTI LEVEL INDEXING

- As the database grows, the index also becomes large and may not fit in main memory, leading to multiple disk accesses.
- Multilevel indexing solves this by breaking the large index into smaller blocks.
- These smaller blocks fit in memory more easily and point to each other in layers, reducing overhead and improving access speed.

MULTI LEVEL INDEXING

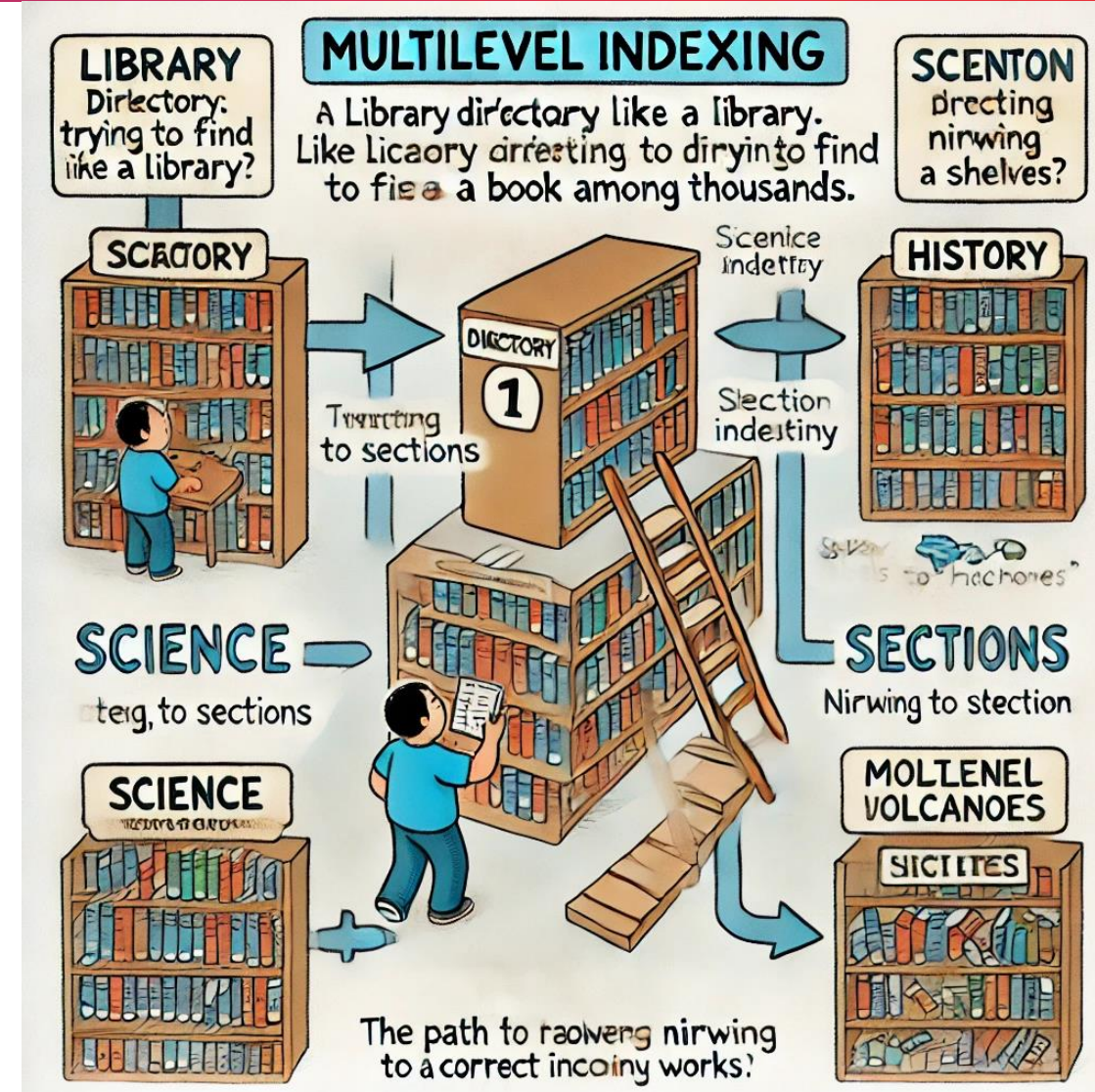
- Multilevel indexes use a hierarchical structure to improve data retrieval in large databases.
- They reduce disk access and speed up queries by organizing indexes in levels, like a search tree.
- They can be built on any first-level index that spans multiple disk blocks. However, updating them is harder since each level must stay ordered.



REAL LIFE - MULTILEVEL INDEXING

Finding Books in a Huge Library

- Imagine walking into a huge library with thousands of books, but there are no signs or labels.
- Finding one specific book would take forever!
- That's why libraries organize books into sections and shelves—to help you find things faster.
- Multilevel indexing in a database works the same way.
- It uses multiple levels to organize data, so the system can find information quickly without checking every single record.



HOW MULTILEVEL INDEXING WORKS IN THE LIBRARY

Think About How A Library Might Help You Find A Science Book About Volcanoes

1. First Level: Library Directory

You go to the Directory at the entrance. It tells you to head to the “Science” section for volcano books. This is like the “first-level” of a multilevel index.

2. Second Level: Section Sign

In the Science section, you find signs pointing to different topics like “Physics,” “Chemistry,” and “Earth Science”. The “Earth Science” sign points you to a specific shelf. This sign helps narrow down your search to just one shelf.

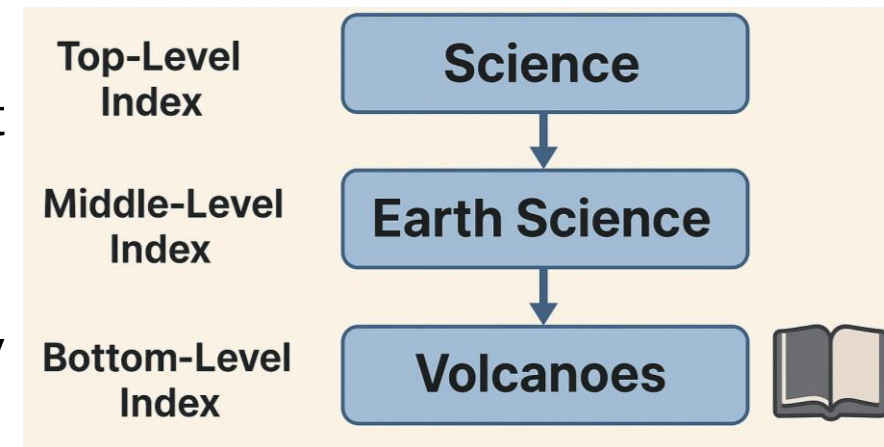
3. Third Level: Shelf Label

On the Earth Science shelf, there are labels on each row that show topics like “Weather,” “Rocks,” and “Volcanoes”. You go to the “Volcanoes” row, and finally, you find your book!

MULTILEVEL INDEXING IN DATABASES

In a database like PostgreSQL, multilevel indexing works the same way to speed up the search. Here's what each level of indexing does:

- **Top-Level Index:**
 - Acts like the library directory, directing you to the right category (all records with “Science”).
- **Middle-Level Index:**
 - Directs you to a smaller section within that category (“Earth Science”).
- **Bottom-Level Index:**
 - Points to the exact row or record (“Volcanoes” book) you want.



Activity

Consider a database with a block size of 64 bytes. It includes a table consisting of 46 rows. Each block pointer occupies 10 bytes. The table is ordered. The structure of the table is as follows:

Id (4B)	Name (8 B)	Gender_id (4B)	Designation_id (4B)
---------	------------	----------------	---------------------

Find the following details:

Part A

- Row size of the table
- Blocking factor of the table
- Number of blocks in the table
- How many rows are there in the 16th block?
- Find the cost of searching a query.

Answer

Part A

i. Row size = $4 + 8 + 4 + 4 = 20 \text{ B}$

ii. Blocking Factor = $\left\lfloor \frac{\text{Block Size}}{\text{Row Size}} \right\rfloor = \left\lfloor \frac{64 \text{ B}}{20 \text{ B}} \right\rfloor = \lfloor 3.2 \rfloor = 3$

iii. # blocks = $\left\lceil \frac{\text{No of Rows}}{\text{Block Factor}} \right\rceil = \left\lceil \frac{46}{3} \right\rceil = \lceil 15.3 \rceil = 16$

iv. Rows in the 16th block = $15 * 3 = 45$
 $= 46 - 45 = 1 \text{ row}$

v. Since the table is ordered, binary search can be applied.

$$\text{cost} = \lfloor \log_2(\# \text{ blocks}) \rfloor + 1 = \lfloor \log_2(16) \rfloor + 1 = \lfloor 4 \rfloor + 1 = 5 \text{ IO}$$

DISCUSSION

Employee

	id	name	gender_id	designation_id
B1				
B2				
B3				
B4				
B16				

3 rows

Activity Contd...

- After applying primary index to the table using id. Find the followings again:

Part B:

- i. Row size of the table.
- ii. Blocking factor of the table.
- iii. Number of blocks in the table.
- iv. Find the new cost for searching a query.

Answer

Part B

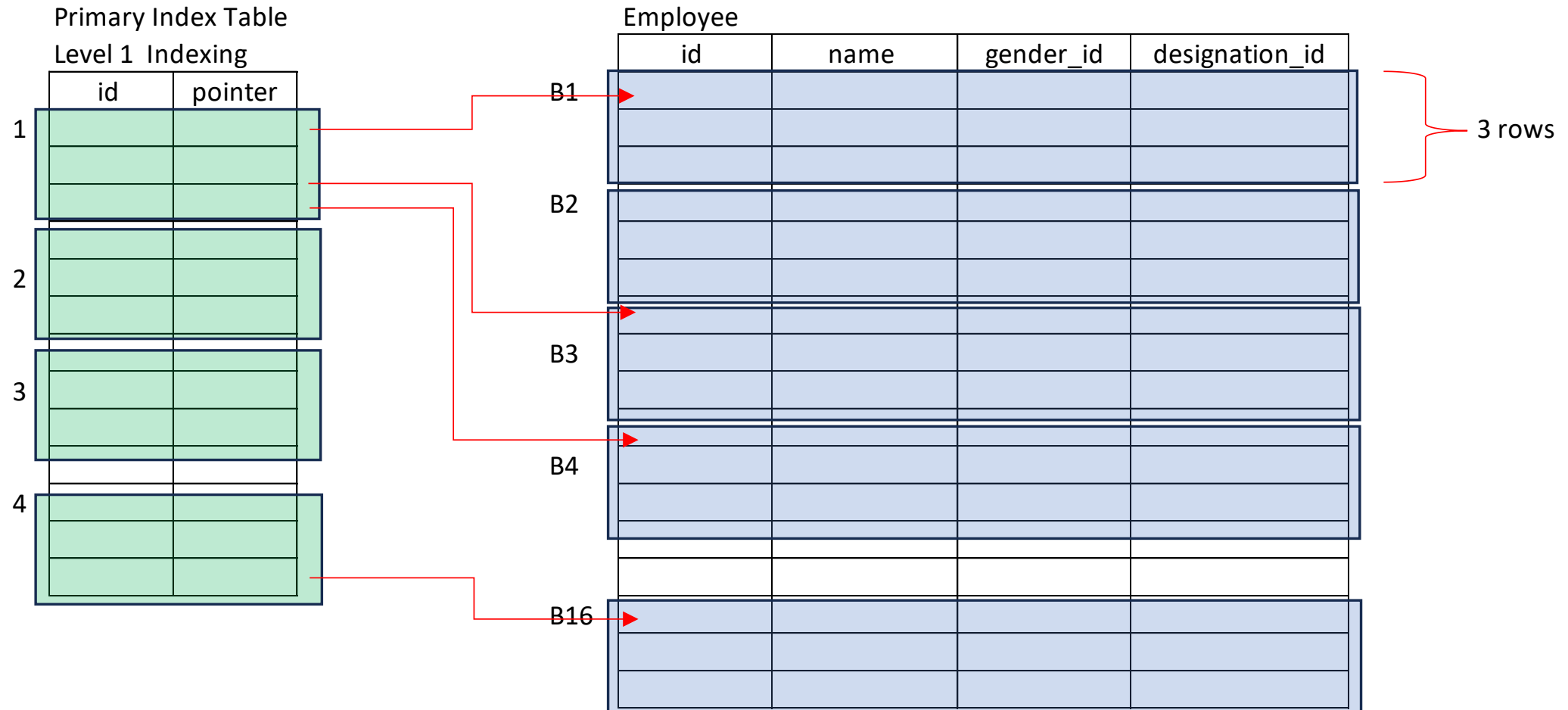
i. Row size = $4 + 10 = 14$ B

ii. Blocking Factor = $\left\lfloor \frac{\text{Block Size}}{\text{Row Size}} \right\rfloor = \left\lfloor \frac{64 \text{ B}}{14 \text{ B}} \right\rfloor = \lfloor 4. \rfloor = 4$

iii. # blocks = $\left\lceil \frac{\text{No of Rows}}{\text{Block Factor}} \right\rceil = \left\lceil \frac{16}{4} \right\rceil = \lceil 4 \rceil = 4$

iv. $\text{cost} = \text{Index table} + \text{Host table} = (\lceil \log_2(4) \rceil + 1) + 1 = (\lceil 2 \rceil + 1) + 1 = 4 \text{ IO}$

DISCUSSION



Activity Contd...

- Let's try to create another index for the primary index table.

NOTE: The row size and the block factor will be same as the primary index table.

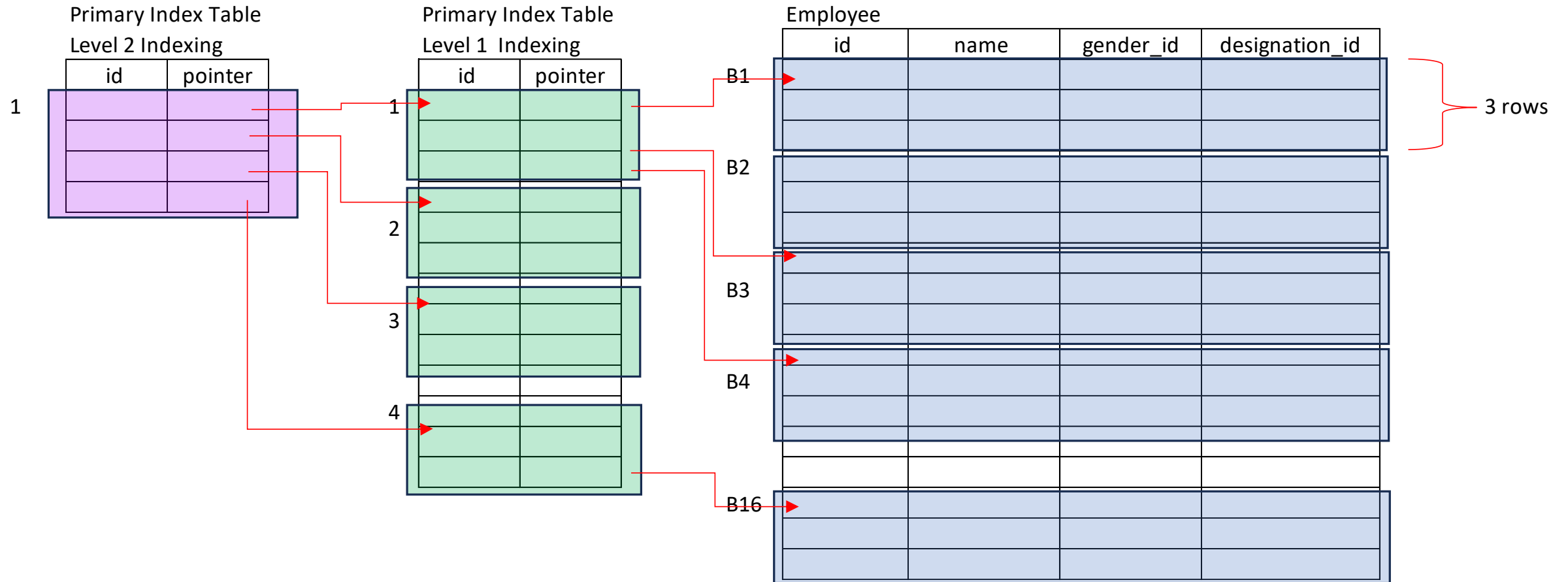
Therefore,

$$\# \text{ blocks} = \left\lceil \frac{\text{No of Rows}}{\text{Block Factor}} \right\rceil = \left\lceil \frac{4}{4} \right\rceil = \lceil 1 \rceil = 1$$

$$\text{cost} = \text{Index table} + \text{Index table} + \text{Host table}$$

$$= (\lceil \log_2(1) \rceil + 1) + 1 + 1 = (\lceil 0 \rceil + 1) + 1 + 1 = 3 \text{ IO}$$

DISCUSSION



Activity

- Consider the employee table containing 60 blocks of data. A cluster index contain 20 records spanning across 6 blocks is created to reduce the cost.
 - a. How many distinct values are there in the indexing column?
 - b. What is the cost, if the index is not used?
 - c. What is the cost if the index is used?
 - d. Is the indexing useful?
 - e. To further reduce the cost another level of index is added. Calculate the new cost?
 - f. Is it worthy to add more levels?

ANSWER

- a. Since there are 20 records in the cluster index. There are 20 distinct values.
- b. 60 IO
- c. Cost = Index table + Host *table*
$$= (\lfloor \log_2(6) \rfloor + 1) + \left\lceil \frac{60}{20} \right\rceil = (\lfloor 2. \rfloor + 1) + \lceil 3 \rceil = 6 \text{ IO}$$
- d. Yes. It is useful because it reduces the cost from 60 IO to 6 IO.

ANSWER

d. After adding another index which is primary index to the cluster index table.

$$\text{Block Factor} = \left\lfloor \frac{\# \text{ of records}}{\# \text{ of blocks}} \right\rfloor = \left\lfloor \frac{20}{6} \right\rfloor = \lfloor 3.3 \rfloor = 3$$

(Cluster index row size is equal to the primary index row size. Therefore, block factors are equal in both. Since the 2nd level index is a primary index, its number of blocks is equivalent to number of blocks in cluster index)

$$\# \text{ blocks} = \left\lceil \frac{6}{3} \right\rceil = 2$$

$$\begin{aligned} \text{Cost} &= \text{Primary Index table} + \text{Cluster Index table} + \text{Host table} \\ &= (\lceil \log_2(2) \rceil + 1) + 1 + \left\lceil \frac{60}{20} \right\rceil = 6 \text{ IO} \end{aligned}$$

f. Adding more levels of indexing is not useful.

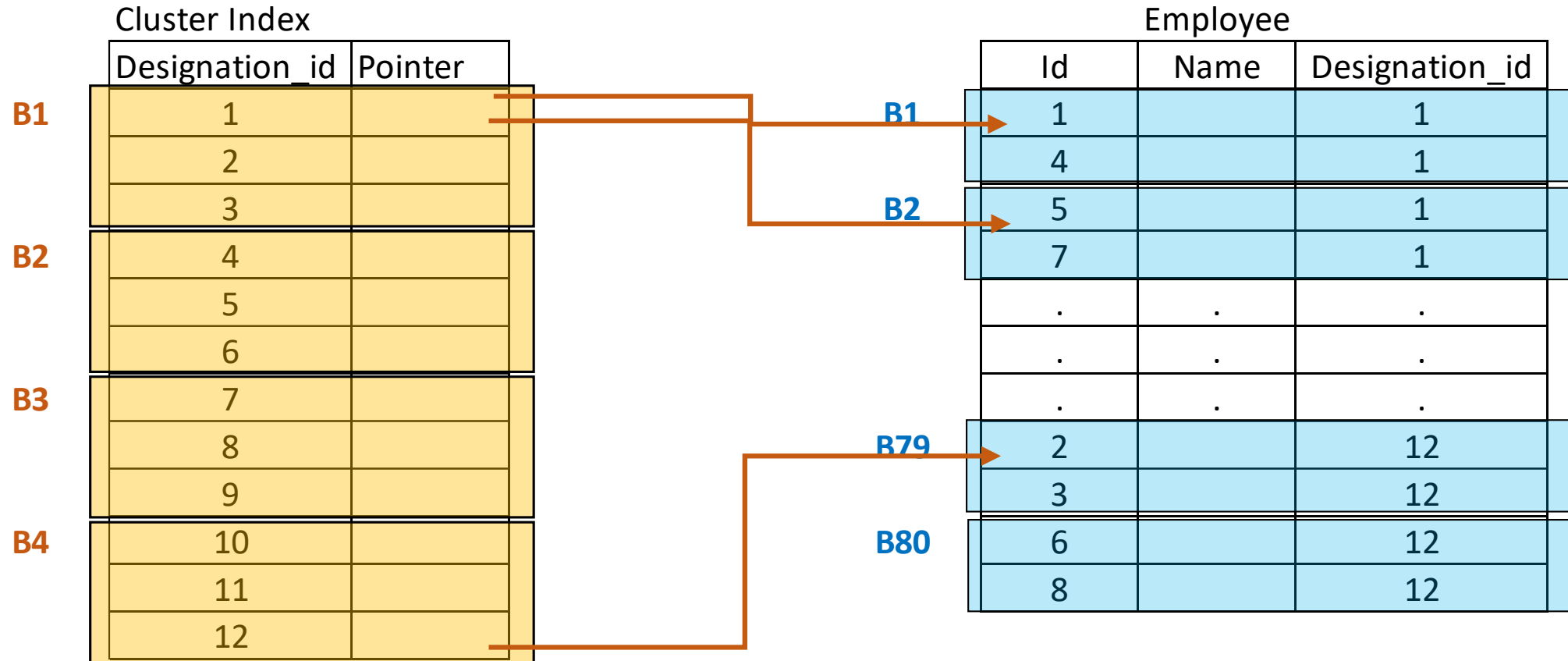
ACTIVITY

Find the cost of a query for the following table:

Employee			
	Id	Name	Designation_id
B1	1		1
	4		1
B2	5		1
	7		1
	.	.	.
	.	.	.
	.	.	.
B79	2		12
	3		12
B80	6		12
	8		12

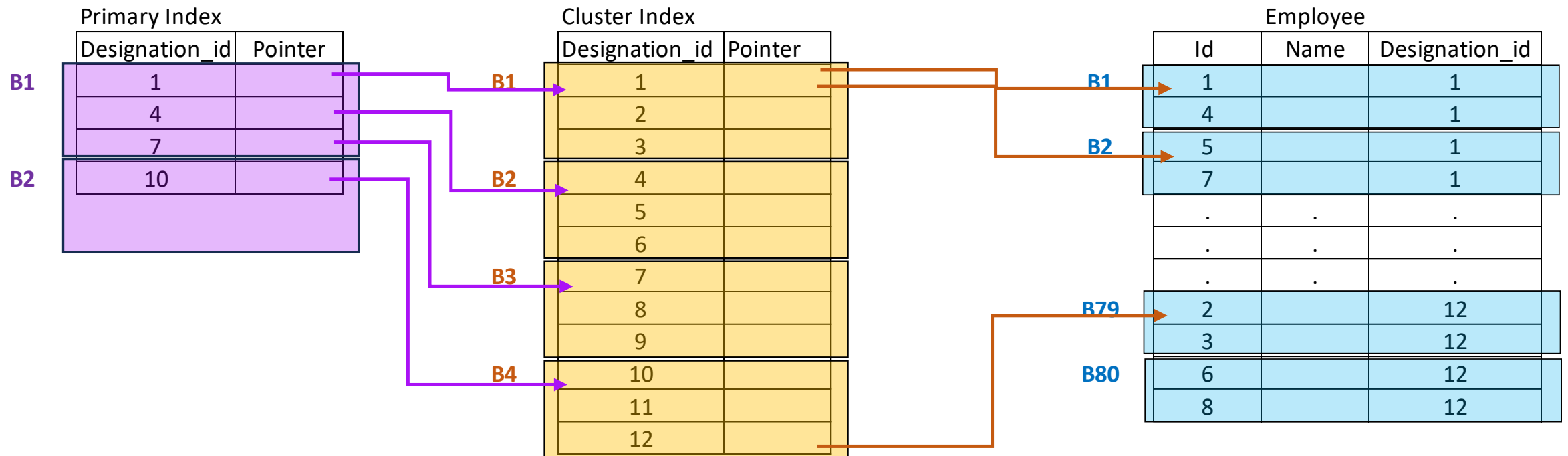
ACTIVITY contd...

- Find the cost of a query for the following table: (Assume that each designation has same number of employees)



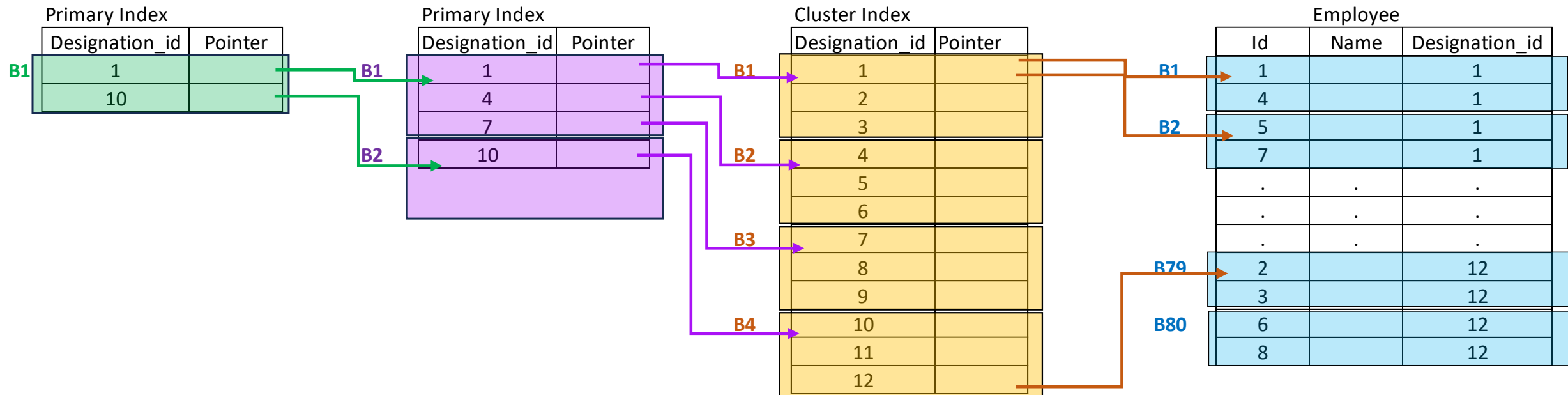
ACTIVITY contd...

- Find the cost of a query for the following table:



ACTIVITY contd...

- Find the cost of a query for the following table:



Activity - Discussion

Table and index design	Cost
Only the base/ host table	
Host table with a clustered index using level-1 indexing	
A primary index pointing to a clustered index and host table (Level-2 indexing)	
A primary index that points to another primary index, which then points to a clustered index and the host table (Level-3 indexing)	

Is adding additional levels of indexing beneficial in this scenario? ? Justify your answer.

ANALYZE THE PERFORMANCE IMPROVEMENT

1. Check Execution Plans

- In PostgreSQL, EXPLAIN ANALYZE shows the steps the database takes to execute a query. After creating the index, the plan should show that the database uses the index instead of scanning the entire table.

2. Compare Cost Reduction

- Without an index, PostgreSQL performs a **sequential scan**, which has a higher I/O cost.
- With an index, PostgreSQL performs an **index scan**, which reduces the cost by narrowing down to relevant records faster.

DISCUSSION AND OBSERVATION

1.Impact of Indexing on Query Performance

- You should observe a substantial reduction in the execution time when using the multilevel index.

2.Cost Breakdown

- Compare the Total Cost and Execution Time from both queries to understand the impact.

CONCLUSION

- This practical session demonstrates how multilevel indexing in PostgreSQL can drastically improve query performance by reducing the number of rows PostgreSQL has to examine.
- By using indexes wisely, you can optimize your database for faster, more efficient data retrieval.

Multi-column Indexing

EXAMPLE

```
SELECT * FROM employee WHERE nic = '100000888V' AND designation_id = 3;
```

There are four (4) approaches to answer this query.

1. Read all records of employee table (manually)
2. Use an index on gender_id (then check all record on gender_id = 1)
3. Use an index on designation_id (then check all records on designation_id = 3)
4. Use a multi-column index on gender_id and designation_id (direct approach)

NOTE: that 4th approach is the most efficient as it directly points to the required block.

EXAMPLE contd...

gender_id	designation_id	pointer
1	2	
1	3	
2	2	
2	3	



This is the section which points to the blocks we need.

Multicolumn Indexing

- Usually an index has two (2) columns. They are index column and block pointer.
- However, to answer certain queries efficiently we can have multiple indexing columns in an index.
- Such indexes are called multi-column indexing.

MULTI-COLUMN INDEXING

- In PostgreSQL, a multi-column index (also known as a composite index) can be created to improve query performance on multiple columns simultaneously.
- Multi-column indexes are useful when queries commonly filter or sort by multiple columns together.

```
CREATE INDEX index_name ON table_name (column1, column2, ...);
```

ACTIVITY 8 – MULTI-COLUMN INDEXING

Activity



1. Execute query and observe the cost of the query,

```
EXPLAIN SELECT * FROM employee WHERE nic = '100000789V' AND  
designatation_id = 3;
```

2. Create Index on designatation_id and nic.

```
CREATE INDEX idx_employee_nic_desig ON employee (designatation_id, nic);
```

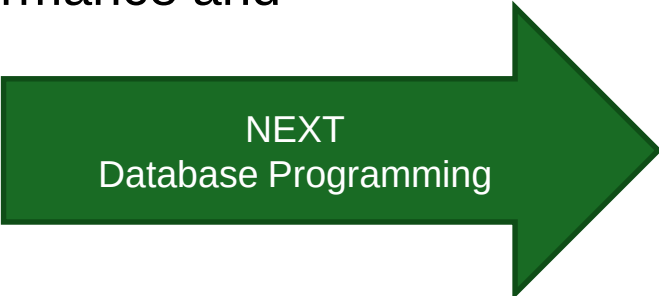
3. Again, execute the same query and compare the results.
4. Check the created indexes using the following statement.

```
SELECT * FROM pg_stat_user_indexes;
```

The PostgreSQL system view pg_stat_user_indexes provides statistics about index usage for user-defined tables (i.e., non-system tables) in a PostgreSQL database.

Summary

- By the end of this section, students should be able to:
- **ILO1:** Identify different types of indexes in PostgreSQL, including primary, cluster, and secondary indexes.
- **ILO2:** Explain the physical file structure of a database and how it impacts row size calculations.
- **ILO3:** Analyze the role of the query planner in optimizing database performance and the use of index-only scans.
- **ILO4:** Demonstrate the process of physical database tuning through efficient indexing strategies, including multi-level and multi-column indexing.
- **ILO5:** Evaluate the implications of unused indexes on database performance and storage efficiency.



NEXT
Database Programming